

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	7
1. СТРУКТУРА КУРСОВОЙ РАБОТЫ.....	9
2. ФОРМУЛИРОВКА ТЕМЫ, ЦЕЛИ И ЗАДАЧ КУРСОВОЙ РАБОТЫ.....	10
2.1. Разработка листа задания	10
2.2. Формулировка темы курсовой работы.....	11
2.3. Формулировка цели курсовой работы.....	13
2.4. Краткое описание ожидаемого результата и структуры работы	13
2.5. Самопроверка по формулировке темы и цели курсовой работы.....	14
3. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА СТЕКА ТЕХНОЛОГИЙ	15
4. СОЗДАНИЕ ПРОТОТИПА ИНТЕРФЕЙСА.....	17
4.1. Введение в процесс создания прототипа	17
4.2. Анализ требований к интерфейсу.....	17
4.3. Инструменты и подходы к прототипированию.....	18
4.3.1. Введение в Figma	18
4.3.2. Интерфейс Figma.....	18
4.3.3. Основы работы с инструментами Figma	19
4.3.4. Создание структуры веб-сайта	19
4.3.5. Прототипирование	19
4.3.6. Совместная работа и экспорт.....	20
4.4. Процесс проектирования прототипа	20
4.5. Документирование и результаты	21
5. РАЗРАБОТКА АРХИТЕКТУРЫ ПРОЕКТА	22
5.1. Основы структурирования проекта по БЭМ	22
5.1.1. Общая идея методологии БЭМ.....	22
5.1.2. Ключевые понятия БЭМ.....	23
5.1.3. Рекомендованная структура папок и файлов	24
5.1.4. Основные принципы ООП в контексте UI-компонентов	25
5.2. Применение модульности и структурирования	29
5.2.1. Принципы модульности	29
5.2.2. Тестирование компонентов	33
5.3. Поддержка адаптивности и масштабируемости архитектуры	33

5.3.1. Использование компонентов в приложении.....	37
5.3.2. Применение БЭМ для адаптивности.....	38
5.3.3. Ключевые моменты адаптивной и масштабируемой архитектуры	39
5.3.4. Документирование архитектуры	40
6. РЕАЛИЗАЦИЯ ПРОЕКТА	41
6.1. Подготовка окружения для разработки.....	41
6.2. Разработка пользовательского интерфейса.....	46
6.3. Реализация функциональности приложения.....	46
6.3.1. Создание структуры приложения, включающей основные страницы или маршруты (роутинг)	47
6.3.2. Разработка компонентов с использованием композиции и инкапсуляции	48
6.3.3. Использование состояния для управления данными	50
6.3.4. Настройка обработки пользовательских событий	55
6.3.5. Реализация динамических обновлений интерфейса	59
6.4. Подключение серверной части и работа с API	63
6.4.1. Настройка запросов к API.....	63
6.4.2. Обработка полученных данных и отображение на странице	66
6.4.3. Работа с REST API или GraphQL для получения и отправки данных .	70
6.4.4. Обработка ошибок и уведомления пользователя.....	74
6.4.5. Реализация авторизации и аутентификации.....	76
6.5. Добавление анимаций и улучшение пользовательского опыта.....	80
6.5.1. Роль анимаций и плавных переходов в UX.....	81
6.5.2. Настройка плавных переходов между страницами или компонентами	82
6.5.3. Улучшение производительности интерфейса (lazy-loading)	83
6.5.4. Общие рекомендации по анимациям и UX	83
7. ТЕСТИРОВАНИЕ И ОТЛАДКА	85
7.1. Тестирование React-приложений.....	85
7.1.1. Юнит-тестирование (Jest).....	85
7.1.2. Тестирование компонентов (React Testing Library)	86
7.1.3. Интеграционные тесты (Cypress)	87
7.2. Оптимизация React-приложений	87
7.2.1. Мемоизация	87

7.2.2. Состояние на всех уровнях	88
7.2.3. Продвинутая навигация.....	89
7.2.4. Ленивая загрузка	90
7.2.5. Организация кода и архитектура	90
7.3. Подготовка проекта к продакшену	91
7.3.1. Строгий режим React	91
7.3.2. Бандлинг и динамический импорт	92
7.3.3. Деплой с CI/CD	92
7.3.4. Аналитика	92
7.4. Дополнительные материалы	93
7.4.1. SSR (Server-Side Rendering)	93
7.4.2. Анимации (React Transition Group)	93
7.4.3. Сборка библиотеки компонентов	94
8. ОБЩИЕ ПОЛОЖЕНИЯ	95
8.1. Цели и задачи курсовой работы.....	95
8.2. Структура и объем курсовой работы.....	95
9. ОФОРМЛЕНИЕ КУРСОВОЙ РАБОТЫ	98
9.1. Построение отчета	99
9.2. Нумерация страниц отчета.....	99
9.3. Нумерация разделов, подразделов, пунктов, подпунктов и книг отчета.	100
9.4. Иллюстрации	101
9.5. Таблицы.....	101
9.6. Ссылки.....	102
9.7. Список использованных источников.....	102
9.8. Приложения	102
9.9. Листинги (программный код)	103
10. ПОРЯДОК ПРЕДСТАВЛЕНИЯ КУРСОВОЙ РАБОТЫ	104
ЗАКЛЮЧЕНИЕ.....	105
СПИСОК ЛИТЕРАТУРЫ.....	106
Приложение 1 Титульный лист	107
Приложение 2 Заявление на тему	108
Приложение 3 Лист задания.....	109
Приложение 4 Пример заполнения титульного листа	110
Приложение 5 Пример заполнения заявления на тему.....	111

Приложение 6 Пример заполнения листа задания.....	112
Сведения об авторах.....	113

ВВЕДЕНИЕ

Фронтенд-разработка представляет собой одну из ключевых областей современного программирования, которая сосредоточена на создании интерфейсов взаимодействия пользователя с веб-приложениями. Курсовая работа включает разработку визуальной и функциональной части приложений, с которыми конечный пользователь взаимодействует напрямую через браузер. Фронтенд объединяет в себе дизайн, пользовательский опыт (UX) и программирование, что делает его уникальной и сложной дисциплиной. В рамках дисциплины «Фронтенд-разработка» изучаются такие технологии, как HTML, CSS и JavaScript, которые формируют базу веб-разработки, а также современные подходы к созданию интерактивных интерфейсов, включая использование TypeScript, библиотек и фреймворков, таких как React, анимации с использованием keyframes, работа с API, и многое другое.

Одной из важных тем, рассматриваемых в рамках курсовой работы, является применение принципов объектно-ориентированного программирования (ООП) в разработке клиентских модулей. Это позволяет проектировать код, который легче поддерживать, тестировать и масштабировать. Кроме того, студенты изучают подходы к созданию одностраничных приложений (SPA), что дает возможность проектировать более быстрые и интерактивные веб-приложения. Также работа включает изучение архитектурных паттернов, которые помогают организовать процесс разработки, улучшая как качество кода, так и взаимодействие между разработчиками. Уделяется внимание современным инструментам разработки, таким как системы управления версиями, автоматизация процессов сборки и тестирования, что особенно важно для подготовки к работе в реальных проектах.

Курсовой проект по дисциплине "Фронтенд-разработка" позволяет студентам закрепить теоретические знания на практике, создавая полноценное веб-приложение. Студенты начинают с анализа требований, разработки технического задания и проектирования структуры приложения. Затем они переходят к реализации дизайна, программированию интерактивных элементов и интеграции клиентской части с серверными компонентами. Завершающим этапом является тестирование и деплой готового приложения.

Итоговый результат курсовой работы должен представлять собой минимально жизнеспособный продукт (MVP), который выполняет заявленные функции и соответствует требованиям технического задания. Студенты не только демонстрируют навыки разработки, но и учатся организовывать проектную работу, следуя современным стандартам индустрии. Курсовая работа становится связующим звеном между теорией и практикой, помогая студентам подготовиться к реальным профессиональным вызовам.

Данное учебно-методическое пособие содержит всю необходимую информацию по разработке, написанию и подготовке к защите курсовой работы по дисциплине «Фронтенд-разработка».

1. СТРУКТУРА КУРСОВОЙ РАБОТЫ

Курсовая работа (КР) представляет собой самостоятельный проект, выполняемый студентом в рамках изучения дисциплины «Фронтенд-разработка». Она нацелена на закрепление теоретических знаний и практических навыков, полученных в процессе обучения, и демонстрирует способность студента решать реальные задачи разработки веб-приложений. Курсовая работа состоит из трёх основных глав, каждая из которых играет важную роль в создании и завершении проекта.

Структура курсовой работы представляет собой последовательность этапов, охватывающих весь процесс разработки – от постановки задачи до получения готового решения.

Курсовая работа как письменная теоретическая работа должна иметь следующую структуру:

- титульный лист;
- задание на курсовую работу;
- реферат;
- содержание (оглавление);
- термины и определения (при необходимости);
- перечень сокращений и обозначений (при необходимости);
- введение;
- основная часть (главы, разделы, подразделы, пункты, подпункты);
- заключение;
- список использованной литературы (источников);
- приложение(-я) (при необходимости, не более 1/3 от общего объема работы).

Общий объем курсовой работы, как правило, не должен быть более 50 страниц.

2. ФОРМУЛИРОВКА ТЕМЫ, ЦЕЛИ И ЗАДАЧ КУРСОВОЙ РАБОТЫ

Первый этап включает выбор темы проекта, ее формулировку и постановку цели и задач. Студенту необходимо определить, какую проблему решает его проект и какие результаты должны быть достигнуты. На этом этапе важно обозначить актуальность выбранной темы, ее практическую значимость и ожидаемую пользу от реализации. Формулировка темы должна быть четкой и лаконичной, а цели – конкретной и измеримой, чтобы её достижение можно было объективно оценить.

Введение, анализ и обоснование выбора технологий является первой и важной частью курсовой работы, поскольку именно в первой главе студент закладывает основу для дальнейшего изложения содержания. В этом разделе должны быть описаны введение, актуальность темы, цель и задачи работы, формулировка темы и цель проекта и прототипа веб-приложения.

2.1. Разработка листа задания

Лист задания является ключевым документом, определяющим требования к проекту. Здесь студенту предстоит описать используемые технологии, а также перечень вопросов, подлежащих разработке, и обязательного графического материала.

Рекомендации к разработке листа задания:

1. Выявите цель создания приложения, его основные характеристики и задачи, которые оно должно решать.

2. Проанализируйте область применения, сферу, в которой будет использоваться приложение, и тип пользователей, для которых оно разрабатывается.

3. Продумайте основные функции, которые будет необходимо детализировать и реализовать в рамках работы. Например, для приложения электронной коммерции это может быть каталог товаров, система корзины и оформление заказов.

4. Уделите внимание требованиям к интерфейсу с точки зрения доступности, адаптивности и эстетичности, а также нефункциональным требованиям (ограничения по производительности, безопасности и надежности приложения, совместимости с различными браузерами и устройствами).

5. Укажите среду разработки и используемые технологии. Определите, какие инструменты, фреймворки и языки программирования будут использованы в проекте.

2.2. Формулировка темы курсовой работы

Выбор темы курсовой работы — это первый и важнейший этап работы, определяющий направление всех последующих действий. В контексте дисциплины «Фронтенд-разработка» тема должна быть связана с разработкой пользовательских интерфейсов и затрагивать аспекты, изученные в рамках курса. Темы могут включать создание одностраничных приложений, адаптивного дизайна, внедрение анимации, работу с API, использование TypeScript, а также реализацию архитектурных паттернов.

Тема должна быть актуальной и практически значимой. Это значит, что курсовая работа должна решать реальную проблему, обеспечивать удобство взаимодействия пользователей с приложением или демонстрировать современные подходы к разработке. Выбор темы должен учитывать уровень знаний студента, сложность задач и личный интерес, чтобы работа не только соответствовала учебным требованиям, но и мотивировала на глубокое погружение в процесс.

Тема должна быть четко и конкретно обозначена. Например, вместо расплывчатого «Создание веб-приложения» следует указать, что проект будет посвящен разработке определенного типа веб-приложения: «Разработка одностраничного приложения для управления задачами с использованием React и TypeScript».

Примеры тем для курсовой работы:

1. Веб-приложение для учета расходов и доходов.
2. Онлайн-магазин с поддержкой каталога товаров и корзины.
3. Веб-приложение для планирования путешествий.
4. SPA для онлайн-заказа еды.
5. Панель управления для IoT-устройств.
6. Система управления проектами для небольших команд.
7. Онлайн-доска для коллективного рисования.
8. Приложение для составления и анализа расписания.
9. Музыкальный плеер с функцией создания плейлистов.
10. Платформа для проведения опросов и голосований.
11. Электронный журнал посещаемости.

12. Сервис бронирования билетов на мероприятия.
13. Веб-приложение для аренды автомобилей.
14. Веб-приложение для планирования семейного бюджета.
15. Приложение для создания и отслеживания привычек.
16. Веб-приложение для поиска работы.
17. Панель администратора для онлайн-курсов.
18. Веб-приложение для составления карт знаний.
19. Приложение для изучения программирования с уровнями сложности.
20. Веб-сервис для аренды недвижимости.
21. Приложение для отслеживания здоровья.
22. Приложение для бронирования столиков в ресторанах.
23. Приложение для поиска репетиторов.
24. Интерфейс для онлайн-конференций.
25. Платформа для продажи подержанных вещей.
26. Приложение для составления кулинарных рецептов.
27. Приложение для отслеживания состояния домашних растений.
28. Онлайн-платформа для организации мероприятий.
29. Приложение для сравнения характеристик товаров.
30. Панель управления для мониторинга активности сайта.
31. Веб-приложение для планирования путешествий с друзьями.
32. Платформа для аренды техники.
33. Приложение для учета калорий и составления диеты.
34. Веб-приложение для анализа данных с сенсоров.
35. Сервис для создания инфографики.
36. Приложение для расчета инвестиционных портфелей.
37. Приложение для создания интерактивных учебных материалов.
38. Веб-приложение для организации турнирной сетки.
39. Приложение для создания виртуальных визиток.
40. Приложение для планирования рабочих смен.
41. Приложение для мониторинга учебной успеваемости.
42. Приложение для визуализации данных о дорожной ситуации.
43. Платформа для проведения хакатонов.
44. Приложение для визуализации генеалогических деревьев.
45. Веб-приложение для организации занятий с репетиторами.
46. Приложение для онлайн-торговли криптовалютой.
47. Приложение для мониторинга состояния сети и серверов.
48. Приложение для управления задачами с функцией напоминаний.

- 49. Веб-приложение для работы с банковскими транзакциями.
- 50. Веб-приложение для управления складами.
- 51. Приложение для автоматизации задач HR-отдела.

2.3. Формулировка цели курсовой работы

Цель курсовой работы формулируется на основе темы и отражает конечный результат, которого нужно достичь. Она должна быть измеримой, конкретной и ориентированной на решение заявленной задачи. Например:

Для темы «Разработка веб-приложения для учета финансов» цель может быть сформулирована как:

«Создать удобное приложение для управления личными финансами с функцией добавления доходов и расходов, визуализацией данных и возможностью экспортировать отчет».

Для темы «Одностраничное приложение для заказа еды» цель может звучать так:

«Разработать одностраничное приложение, позволяющее пользователям выбирать блюда из каталога, добавлять их в корзину и оформлять заказ через интеграцию с REST API».

Важно, чтобы цель была достижима в рамках выделенного времени и учитывала ресурсы, доступные студенту. Цель должна описывать ожидаемый результат.

2.4. Краткое описание ожидаемого результата и структуры работы

Необходимо описать, чего ожидается достичь по итогам работы, а также связь темы, цели и задач. Тема задает общее направление работы, цель описывает, какой результат требуется получить, а задачи помогают поэтапно достичь этой цели. Например:

Тема: «Разработка веб-приложения для управления привычками».

Цель: «Создать интерактивное приложение для отслеживания пользовательских привычек».

Задачи:

1. Реализовать интерфейс для добавления и редактирования привычек.
2. Разработать систему напоминаний и аналитики прогресса.
3. Обеспечить адаптивный дизайн и кроссбраузерную совместимость.

2.5. Самопроверка по формулировке темы и цели курсовой работы

После формулировки темы, цели и задач важно проверить их на соответствие следующим критериям:

1. Насколько тема соответствует дисциплине и изученным темам?
2. Насколько конкретно и четко сформулирована цель?
3. Реальна ли цель с учетом доступного времени и ресурсов?

Только после согласования темы и цели с преподавателем можно переходить к разработке детального плана и выполнению работы.

3. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА СТЕКА ТЕХНОЛОГИЙ

Для написания анализа предметной области студенту необходимо организовать работу по изучению современных технологий разработки клиентских интерфейсов, акцентируя внимание на подходах, которые активно применяются в сфере фронтенд-разработки. Главная задача – провести всесторонний анализ инструментов и методов, используемых в индустрии, и продемонстрировать их значимость для выполнения курсового проекта.

Начать необходимо с общей характеристики современных требований к клиентским интерфейсам. Важно отразить актуальность темы и пояснить, почему в настоящее время большое внимание уделяется разработке удобных, производительных и интуитивно понятных пользовательских интерфейсов.

Далее необходимо перейти к описанию ключевых принципов, лежащих в основе создания фронтенд-приложения. Уделите внимание концепциям разделения логики клиентской части, роутингу внутри приложения, применению виртуального DOM и управлению состоянием. При написании этого раздела важно опираться на официальную документацию или авторитетные источники, чтобы использовать проверенную информацию.

Третий этап раздела предполагает обзор популярных фреймворков, таких как React, Angular и Vue.js. Ваша цель – выделить их основные особенности, сильные и слабые стороны, а также сравнить их по ключевым параметрам, например, по производительности, удобству изучения и возможностям масштабирования. Для большей полноты анализа можно включить в обзор и менее известные, но перспективные фреймворки, которые используют современные подходы.

В отдельном разделе обратите внимание на роль TypeScript в разработке. Поясните, почему его использование считается стандартом для масштабируемых проектов, и приведите примеры интеграции TypeScript с различными фреймворками. Здесь также можно подчеркнуть, как типизация способствует снижению числа ошибок и упрощает командную разработку.

Особое внимание уделяется тому, чтобы студент привел сравнительный анализ технологий, которые могли бы использоваться для аналогичных задач. Например, если проект требует создания SPA-приложения, студент может рассмотреть такие фреймворки, как React, Angular и Vue.js, и объяснить, почему выбран конкретный инструмент. В сравнении важно учитывать такие параметры, как производительность, простота изучения, популярность в сообществе разработчиков и наличие готовых библиотек.

Отдельное внимание следует уделить совместимости выбранных инструментов. Студент должен указать, как выбранные технологии будут взаимодействовать друг с другом в рамках проекта.

Заключительную часть главы необходимо посвятить итогам выбора. Здесь студенту следует показать, как выбранные технологии соответствуют поставленным целям и задачам. Важно акцентировать внимание на том, как использование данных инструментов ускорит процесс разработки, повысит производительность приложения и упростит его поддержку в будущем.

В завершение рекомендуется упомянуть о возможных рисках, связанных с выбранными инструментами. Например, если используется новая или редкая технология, следует указать, какие шаги будут предприняты для минимизации рисков (например, изучение документации, прохождение онлайн-курсов). Такой подход покажет, что студент не только осознанно подходит к выбору, но и способен предвидеть возможные проблемы. Укажите, какие подходы и технологии кажутся наиболее перспективными для разработки, и обоснуйте свой выбор с учетом требований курсовой работы. Главное – показать осознанный и аналитический подход к выбору инструментов, а не просто перечислить их возможности.

4. СОЗДАНИЕ ПРОТОТИПА ИНТЕРФЕЙСА

Создание прототипа интерфейса – это один из ключевых этапов разработки клиентских приложений. Эта часть работы демонстрирует, как студент планирует взаимодействие пользователя с системой, применяя основные принципы UX/UI-дизайна. Студенты должны показать, что умеют создавать функциональные и удобные интерфейсы, которые соответствуют целям проекта и требованиям, описанным в листе задания.

На этапе создания прототипа студент разрабатывает визуальное представление будущего веб-приложения. Это позволяет проверить, насколько удобно и логично будет взаимодействие пользователя с интерфейсом. Прототипирование включает проектирование макетов страниц, размещение основных элементов интерфейса и настройку навигации. Современные инструменты, такие как Figma или Adobe XD, помогают создать качественный и детализированный прототип, который станет основой для дальнейшей реализации.

4.1. Введение в процесс создания прототипа

Начать главу следует с объяснения важности прототипирования на этапе разработки клиентского приложения. Студенты могут отметить, что прототип интерфейса — это инструмент, который позволяет визуализировать структуру и функциональность приложения, протестировать пользовательский опыт до начала реализации и выявить возможные недочеты на ранних этапах. Важно подчеркнуть, что создание прототипа помогает сократить время разработки, снизить издержки и улучшить качество конечного продукта.

4.2. Анализ требований к интерфейсу

В этой части содержится анализ требований, сформулированных в техническом задании. Студенты должны рассмотреть, какие элементы интерфейса необходимы для реализации функциональных возможностей системы. Например, если проект связан с системой управления задачами, то интерфейс должен включать:

1. Панель навигации.
2. Список задач с фильтрацией.

3. Форму для добавления новых задач.

4. Статистику выполнения заданий.

Здесь важно выделить не только функциональные элементы, но и особенности пользовательского опыта. Например, студенты могут указать, что интерфейс должен быть адаптивным, чтобы поддерживать использование на разных устройствах (десктоп, планшет, смартфон).

4.3. Инструменты и подходы к прототипированию

Далее следует описать инструменты, которые будут использоваться для создания прототипа. Студенты могут выбрать специализированное ПО (например, Figma) или встроенные инструменты фреймворков (например, Storybook для React). Важно указать, почему был выбран тот или иной инструмент и как он соответствует поставленным задачам.

Ниже представлена примерная пошаговая инструкция по использованию Figma при создании макета (дизайн-макета) собственного веб-приложения. Данные рекомендации могут быть адаптированы в зависимости от специфики курса и заданий.

4.3.1. Введение в Figma

Figma – это мощный инструмент для дизайна и прототипирования, который позволяет создавать интерактивные макеты веб-сайтов и приложений.

Для начала работы с Figma необходимо зарегистрироваться на официальном сайте: <https://www.figma.com/> [2]. После регистрации вы получите доступ к облачному редактору, который не требует установки на компьютер.

4.3.2. Интерфейс Figma

Интерфейс Figma состоит из нескольких ключевых элементов:

1. Панель инструментов (слева) – содержит инструменты для создания фигур, текста, векторных элементов и т.д.
2. Слои (слева) – отображает структуру проекта.
3. Свойства (справа) – позволяет настраивать параметры выбранного элемента.
4. Рабочая область (центр) – пространство для создания макетов.

Чтобы начать работу, создайте новый файл, нажав на кнопку "New Design File". Figma поддерживает несколько типов файлов, включая дизайн, прототип и презентацию.

4.3.3. Основы работы с инструментами Figma

1. Работа с фигурами и текстом.

Figma предоставляет базовые инструменты для создания фигур (прямоугольники, круги, линии) и текста. Чтобы добавить элемент, выберите соответствующий инструмент на панели слева и нарисуйте его на рабочей области.

2. Использование сеток и направляющих.

Для точного выравнивания элементов используйте сетки и направляющие. Включить сетку можно через меню View > Show Grid. Это особенно полезно при создании адаптивных макетов.

3. Работа со слоями.

Каждый элемент в Figma – это слой. Управляйте слоями через панель слева: переименовывайте, группируйте и изменяйте порядок. Это помогает поддерживать порядок в проекте.

4.3.4. Создание структуры веб-сайта

1. Проектирование макета.

Начните с создания основных блоков веб-сайта:

- шапка (Header) – содержит логотип и навигацию;
- основной контент (Main Content) – включает тексты, изображения и другие элементы;
- футер (Footer) – содержит контактную информацию и ссылки.

Используйте прямоугольники и текстовые блоки для обозначения этих областей.

2. Добавление интерактивных элементов.

Создайте кнопки, ссылки и другие интерактивные элементы.

4.3.5. Прототипирование

1. Создание связей между фреймами.

Figma позволяет создавать интерактивные прототипы. Для этого:

- выделите элемент (например, кнопку);
- перейдите на вкладку Prototype в правой панели;
- перетащите стрелку на целевой фрейм (например, страницу регистрации).

2. Настройка анимаций.

Вы можете настроить переходы между фреймами, выбрав тип анимации (например, Slide In или Fade) и длительность.

4.3.6. Совместная работа и экспорт

1. Совместная работа.

Figma поддерживает совместную работу в реальном времени. Чтобы поделиться проектом, нажмите кнопку Share в правом верхнем углу и скопируйте ссылку.

2. Экспорт макета.

Для экспорта макета выберите нужный фрейм и нажмите Export в правой панели. Figma поддерживает экспорт в форматы PNG, JPEG, SVG и PDF.

Подробнее изучить основы работы с инструментом Figma студенты могут в материалах [3, 4, 5].

4.4. Процесс проектирования прототипа

Основная часть должна быть посвящена описанию этапов проектирования прототипа. Студенты должны объяснить:

1. Как создавались основные структуры интерфейса (например, размещение хедера, основного контента и футера).

2. Как определялись ключевые элементы взаимодействия пользователя с системой (кнопки, формы, выпадающие списки).

3. Как решались вопросы адаптивности интерфейса (например, использование медиазапросов и гибких сеток).

4. Как прорабатывался дизайн, чтобы соответствовать принципам UX/UI (например, логичное расположение элементов, выбор шрифтов, цветовых схем и кнопок).

Студенты должны упомянуть, что на этапе создания прототипа необходимо учитывать специфику целевой аудитории. Например, если приложение рассчитано на людей старшего возраста, шрифты и кнопки должны быть крупнее, а цветовая гамма – контрастнее.

4.5. Документирование и результаты

Заключительная часть главы должна быть посвящена документированию созданного прототипа. Студенты должны пояснить, какие макеты были созданы (например, макеты для десктопной и мобильной версии приложения) и как они связаны между собой.

5. РАЗРАБОТКА АРХИТЕКТУРЫ ПРОЕКТА

Разработка архитектуры проекта фокусируется на структурировании проекта и определении ключевых компонентов, что является основой для успешной реализации клиентского приложения. Этот этап необходим для создания понятной, поддерживаемой и масштабируемой структуры, которая упрощает дальнейшую разработку, тестирование и интеграцию.

Важно отметить, что правильно спроектированная архитектура позволяет:

1. Обеспечить модульность и повторное использование кода.
2. Упростить поддержку проекта и внесение изменений.
3. Способствовать четкому разделению ответственности между компонентами.

5.1. Основы структурирования проекта по БЭМ

Студентам нужно рассказать о структуре проекта и обосновать выбор технологии БЭМ для организации файлов и папок. Важные аспекты, которые следует раскрыть:

1. Блоки – независимые функциональные элементы интерфейса, например, кнопки, формы, карточки.
2. Элементы – составные части блоков, которые не имеют самостоятельного значения (например, заголовок внутри карточки или кнопка в форме).
3. Модификаторы – описывают состояния или вариации блоков и элементов (например, `button--disabled` или `card--highlighted`).

Рекомендуется выделить базовые папки проекта:

- `/src` – содержит исходный код;
- `/src/blocks` – содержит файлы блоков;
- `/src/styles` – общие стили;
- `/src/assets` – медиафайлы, шрифты и другие ресурсы.

5.1.1. Общая идея методологии БЭМ

БЭМ (Блок, Элемент, Модификатор) – это методология разработки интерфейсов, основанная на разделении кода на независимые и легко переиспользуемые компоненты. В контексте организационной структуры проекта БЭМ предлагает правила, которые помогают:

1. Локализовать стили внутри конкретных компонентов, избегая конфликтов.

2. Чётко обозначать назначение CSS-классов (блоки, элементы и их состояния).

3. Облегчить поддержку и масштабирование кода, поскольку каждый компонент (блок) имеет собственную структуру и не «захламляет» пространство имён других частей проекта.

5.1.2. Ключевые понятия БЭМ

1. Блоки.

Блок – это самостоятельный, смысловой и функциональный элемент интерфейса. Примеры блоков: кнопка (button), форма (form), карточка (card), меню (nav), шапка сайта (header).

Характеристики:

- может использоваться повторно в разных частях приложения;
- не зависит от внешних элементов, а стили и функционал блоков стараются хранить внутри самих блоков;
- название блока обычно отражает его назначение (например, button, card, form).

2. Элементы.

Элемент – это часть блока, которая не имеет самостоятельного значения вне контекста этого блока. Например, в блоке «карточка» элементом может быть заголовок (card_title) или кнопка (card_button), которая визуально и функционально относится именно к карточке.

Характеристики:

- элемент всегда пишется с указанием блока, к которому он принадлежит (через «два подчёркивания» в классах: card_title);
- название элемента обычно отражает роль внутри блока (кнопка в карточке, иконка, заголовок и т.д.).

3. Модификаторы.

Модификатор описывает состояние или вариацию блока или элемента (например, «активный», «с ошибкой», «увеличенный размер»).

Характеристики:

- в классах используется дополнительный суффикс (часто --), например, button--disabled или card_title—highlighted;

— применение модификатора не меняет смысл блока, а лишь уточняет или изменяет его внешний вид или поведение (цвет, размер, состояние).

Схематический пример именования:

- button (блок);
- button__icon (элемент);
- button--large (модификатор блока);
- button__icon--small (модификатор элемента).

5.1.3. Рекомендованная структура папок и файлов

/src – папка с исходными файлами проекта

В современных проектах принято выделять директорий src (source) для хранения всей основной логики и представления. В корне /src находятся точка входа в приложение, настройки роутинга, глобальные провайдеры и т.п. Внутри src может находиться:

/src/blocks – директория с файлами блоков по методологии БЭМ, каждая сущность (блок) размещается в отдельной папке с говорящим названием (button, card, form). В папке блока обычно хранится файл со стилями (.css), компонент (например, .js для React), а при необходимости – тесты (.test.js).

/src/styles – общие стили (переменные, глобальные стили, миксины и т.п.). Импортируйте эти файлы в вашем главном стиле (например, base.css) или непосредственно в компоненты, в зависимости от настроек сборщика (Webpack, Vite и т.д.).

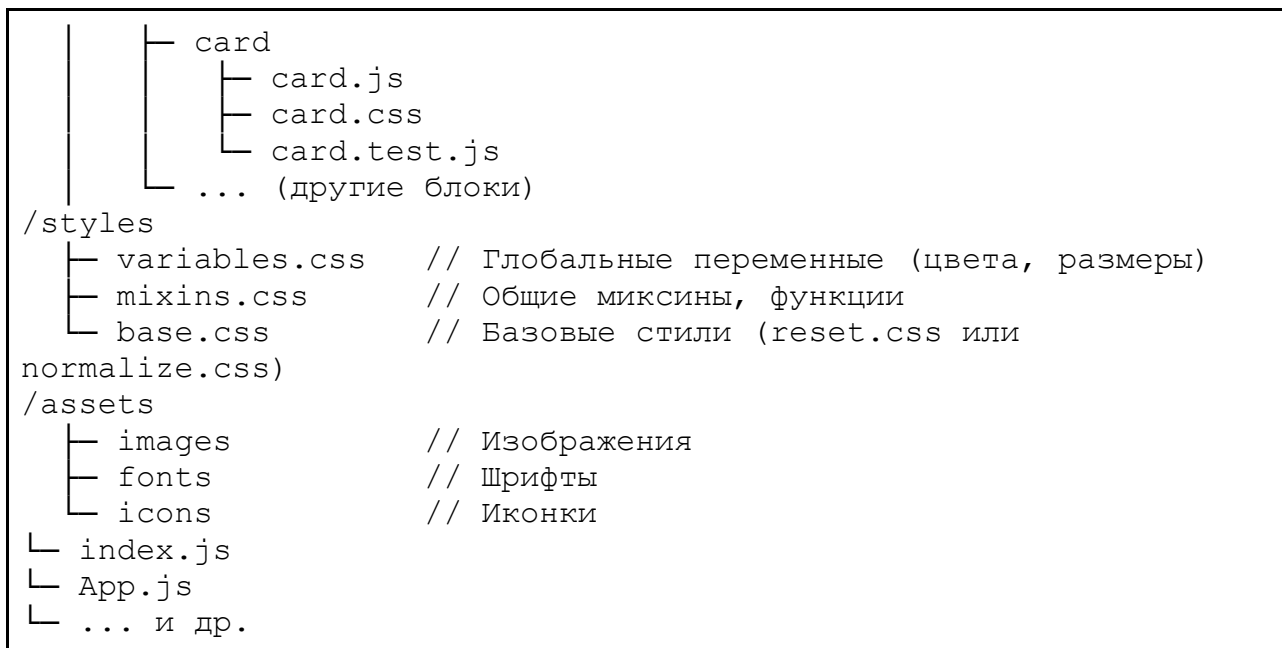
/src/assets – шрифты, изображения, иконки, файлы SVG и прочие ресурсы, необходимые для приложения.

Такое разделение даёт понять, где искать конкретный вид кода: стили блоков – в blocks, общие глобальные стили – в styles, все картинки – в assets.

Внутри папки можно организовать файлы следующим образом (как пример листинг 5.1):

Листинг 5.1. Структура папок и файлов по БЭМ

```
/src
├── /blocks
│   └── button
│       ├── button.js           // Логика блока (например, компонент
│                               // React/Vue)
│       ├── button.css         // Стили блока
│       └── button.test.js      // Тесты на блок (при необходимости)
```



Методология БЭМ помогает поддерживать порядок в структуре файлов и папок, а также делает CSS-код прозрачным и легко масштабируемым. Рекомендуемая структура проекта с папкой `src/blocks` (для отдельных блоков), `src/styles` (для общих стилей) и `src/assets` (для ресурсов) укрепляет модульный подход к разработке.

При использовании БЭМ важно следить за корректностью именования (чтобы не было смешения понятий «блок», «элемент» и «модификатор») и не злоупотреблять модификаторами, вводя их только для реальных вариаций или состояний.

5.1.4. Основные принципы ООП в контексте UI-компонентов

Создание компонентов интерфейса с использованием принципов объектно-ориентированного программирования (ООП), которое позволяет:

1. Создавать повторно используемые элементы.
2. Упрощать расширение функциональности.
3. Строго разделять данные, логику и отображение.

Основные принципы ООП:

1. Инкапсуляция – создание классов компонентов с четко определенными методами и свойствами. Например, кнопка может иметь методы `onClick()` и свойства `isDisabled`.

Пример инкапсуляции представлен на листинге 5.2:

Листинг 5.2. Пример инкапсуляции

```
class Button {
  constructor(label, isDisabled = false) {
    this.label = label;
    this.isDisabled = isDisabled;
  }

  // Публичный метод, который отрисовывает кнопку
  render() {
    // Возвращаем строку HTML или DOM-элемент
    return `
      <button ${this.isDisabled ? 'disabled' : ''}>
        ${this.label}
      </button>
    `;
  }

  // Приватный метод (вне класса может считаться соглашением,
  т.к. в JS нет нативного private для полей в ES5/ES6)
  _handleClick() {
    console.log('Кнопка нажата!');
  }
}
```

2. Наследование – создание более сложных компонентов на основе базовых. Например, компонент "кнопка" (Button) может стать основой для "кнопки-ссылки".

Пример наследования представлен на листинге 5.3:

Листинг 5.3. Пример наследования

```
// Базовый класс кнопки
class Button {
  constructor(label) {
    this.label = label;
  }
  render() {
    return `<button>${this.label}</button>`;
  }
}

// Потомок, который выглядит как кнопка, но ведёт себя как ссылка
class LinkButton extends Button {
  constructor(label, url) {
    super(label);
    this.url = url;
  }
}
```



```

render() {
  // Переопределяем способ рендера
  return `\${this.label}</a>`;
}
}

```

3. Полиморфизм – использование одинаковых методов для компонентов, выполняющих разные действия. Например, метод `render()` может быть определен по-разному для списка задач и отдельной карточки.

Пример полиморфизма представлен на листинге 5.4:

Листинг 5.4. Пример полиморфизма

```

class BaseComponent {
  render() {
    // Базовая реализация рендера
    return '<div>Base Component</div>';
  }
}

class CardComponent extends BaseComponent {
  constructor(title, content) {
    super();
    this.title = title;
    this.content = content;
  }
  render() {
    // Полиморфное поведение
    return `
      <div class="card">
        <h3>${this.title}</h3>
        <p>${this.content}</p>
      </div>
    `;
  }
}

class ListComponent extends BaseComponent {
  constructor(items = []) {
    super();
    this.items = items;
  }
  render() {
    const listItems = this.items.map((item) =>
    `<li>${item}</li>`).join('');
    return `<ul>${listItems}</ul>`;
  }
}

```

```
// При вызове render у потомков будет выполняться их собственная
логика
const card = new CardComponent('Заголовок', 'Текст карточки');
console.log(card.render());
const list = new ListComponent(['Элемент 1', 'Элемент 2']);
console.log(list.render());
```

Организация кода с классами компонентов:

1. Отдельный класс для каждого UI-элемента (блока или модуля). Если вы используете принцип БЭМ, можно создать класс Card, класс Button, класс Form и т.д. Каждый класс хранит логику отрисовки и поведения, инкапсулируя детали реализации.

2. Поддерживать иерархию, близкую к структуре интерфейса. Например, класс Page может включать в себя другие компоненты (классы Header, Footer, Content), создавая дерево объектов, соответствующих дереву DOM.

3. Отдельные файлы для каждого класса. Рекомендуется хранить каждый класс компонента в отдельном файле, что облегчает навигацию и поддержание порядка в структуре проекта.

Пример структуры проекта с классами на листинге 5.5:

Листинг 5.5. Пример структуры проекта с классами

```
/src
├── components
│   ├── BaseComponent.js    // Базовый класс для всех
компонентов
│   ├── Button.js           // Класс Button
│   └── LinkButton.js       // Класс LinkButton (наследуется от
Button)
├── Card.js                 // Класс CardComponent
├── List.js                 // Класс ListComponent
├── pages
│   └── MainPage.js         // Класс MainPage, где создаём
экземпляры компонентов
├── ...
└── index.js
```

Таким образом, необходимо объяснить, как вы структурировали код компонентов, например, создавая отдельные классы для каждого блока или элемента интерфейса.

5.2. Применение модульности и структурирования

Далее следует объяснить, как реализуется модульность кода. Студенты должны продемонстрировать, что каждый компонент (например, форма, карточка, кнопка) выделяется в отдельный модуль, что позволяет подключать и тестировать компоненты независимо друг от друга и переиспользовать компоненты в разных частях приложения.

Каждый модуль должен быть самостоятельным и включать только те зависимости, которые ему нужны.

5.2.1. Принципы модульности

К принципам модульности относятся:

1. Изоляция и переиспользование. Каждый модуль решает конкретную задачу или описывает конкретный элемент интерфейса (компонент). Его код не смешан с кодом других компонентов, что позволяет легко переиспользовать этот модуль в разных местах приложения.

2. Минимальные зависимости. Модуль должен включать только те пакеты и ресурсы, которые ему действительно нужны. Это упрощает поддержку и исключает «паразитные» зависимости, увеличивающие размер приложения.

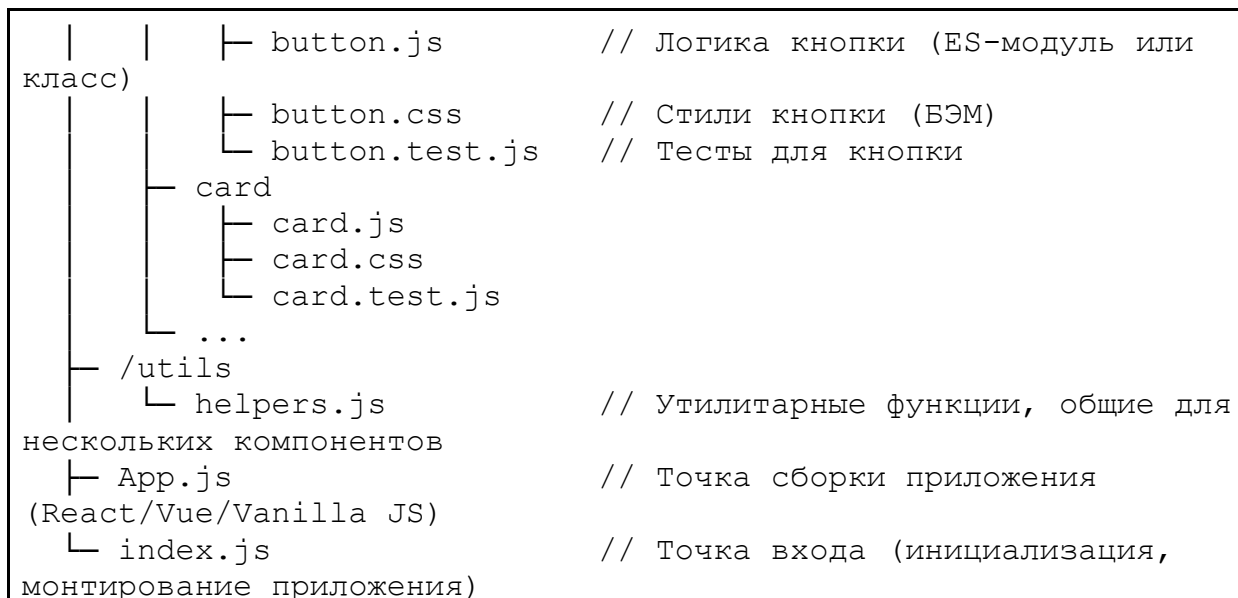
3. Возможность самостоятельного тестирования. Компоненты, выделенные в модули, легче тестировать независимо. Вы можете создать «мок» (имитацию) необходимых данных или окружения и проверить, корректно ли ведёт себя модуль в разных ситуациях.

4. Чистая архитектура. При использовании БЭМ-модулей и/или классов (ООП-подход) каждый компонент имеет собственную структуру файлов (стили, скрипты, тесты). При этом на верхнем уровне приложения вы лишь «собираете» компоненты вместе и описываете логику их взаимодействия.

Рассмотрим, как может выглядеть структура проекта, где каждый блок/компонент хранится в отдельном каталоге вместе со своими зависимостями (листинг 5.6):

Листинг 5.6. Структурирование проекта

```
/src
├── /blocks
│   └── button
```



В папке каждого блока/компонента лежат файлы, относящиеся только к этому компоненту:

- JavaScript/TypeScript-файл с логикой;
- CSS/SCSS/LESS-файл со стилями по БЭМ;
- тесты, проверяющие поведение компонента;
- общие утилиты (например, функции форматирования даты)

располагаются в /utils, а глобальные стили (переменные, миксины) – в /styles.

Для подключения и использования модулей приведем примеры компонентного подхода на React.

Пример для Button.jsx (листинг 5.7):

Листинг 5.7. React-компонент Button

```

// Button.jsx
import React from 'react';
import './button.css'; // стили по БЭМ

export default function Button({ label, onClick }) {
  return <button className="button"
onClick={onClick}>{label}</button>;
}
  
```

Пример для Card.jsx (листинг 5.8):

Листинг 5.8. React-компонент Card

```
// Card.jsx
import React from 'react';
import './card.css';
import Button from '../button/Button';

export default function Card({ title, content }) {
  return (
    <div className="card">
      <h3 className="card__title">{title}</h3>
      <p className="card__content">{content}</p>
      <Button label="Подробнее" onClick={() =>
        console.log('Нажато!')} />
    </div>
  );
}
```

Пример для App.jsx (листинг 5.9):

Листинг 5.9. React-компонент App

```
// App.jsx
import React from 'react';
import Card from './blocks/card/Card';

function App() {
  return (
    <div>
      <Card title="Пример" content="Контент для карточки" />
    </div>
  );
}

export default App;
```

Также в качестве примера представлен пример с ES-модулями (стандарт JavaScript).

Каждый компонент экспортирует нужный интерфейс (например, класс или функцию) с помощью `export`. В другом файле компонент можно импортировать с помощью `import`.

Пример для button.js (листинг 5.10):

Листинг 5.10. Экспорт класса Button

```
// button.js
export class Button {
```

```

    constructor(label) {
        this.label = label;
    }
    render() {
        return `<button>${this.label}</button>`;
    }
}

```

Пример для card.js (листинг 5.11):

Листинг 5.11. Экспорт класса Card

```

// card.js
import { Button } from '../button/button.js';

export class Card {
    constructor(title, content) {
        this.title = title;
        this.content = content;
        this.button = new Button('Подробнее');
    }
    render() {
        return `
            <div class="card">
                <h3 class="card__title">${this.title}</h3>
                <p class="card__content">${this.content}</p>
                ${this.button.render()}
            </div>
        `;
    }
}

```

Файл сборки App.js (листинг 5.12):

Листинг 5.12. Импорт класса Card

```

// App.js
import { Card } from './blocks/card/card.js';

const card = new Card('Заголовок', 'Описание');
document.getElementById('root').innerHTML = card.render();

```

Таким образом, каждый компонент легко подключать и использовать в других местах, а его стили (благодаря БЭМ) не конфликтуют с другими компонентами.

5.2.2. Тестирование компонентов

Автономность компонентов упрощает написание тестов: для каждого компонента (button.js, card.js) есть свой тест (button.test.js, card.test.js).

В тесте создаётся экземпляр компонента, задаются параметры, проверяется ожидаемое поведение.

Пример теста на Jest (JavaScript) представлен в листинге 5.13:

Листинг 5.13. Тестирование компонента Button

```
// button.test.js
import { Button } from './button';

describe('Button component', () => {
  it('должен рендерить кнопку с заданным текстом', () => {
    const btn = new Button('Клик');
    const html = btn.render();
    expect(html).toContain('Клик');
  });

  it('должен быть активен по умолчанию', () => {
    const btn = new Button('Клик');
    const html = btn.render();
    expect(html).not.toContain('disabled');
  });
});
```

Таким образом, студенты демонстрируют, что они выделяют каждый блок, элемент или компонент в свой собственный модуль, который можно подключать, тестировать и переиспользовать независимо.

5.3. Поддержка адаптивности и масштабируемости архитектуры

Важным аспектом является учет требований адаптивности интерфейса и масштабируемости проекта. Студенты должны продемонстрировать, как они заложили возможность добавления новых компонентов и изменения существующих без необходимости переписывать большую часть кода.

Примером может быть создание базового класса для всех интерактивных элементов и наследование от него специфических компонентов (например, кнопок, переключателей).

Ниже приведён пример на «чистом» JavaScript (ES-модули). В реальном проекте вы можете адаптировать его под React-классы или любые другие фреймворки – идея остаётся прежней.

Базовый класс интерактивного элемента (листинг 5.14):

Листинг 5.14. Базовый класс интерактивного элемента

```
// BaseInteractiveElement.js
export class BaseInteractiveElement {
  constructor({ id, className = '', isDisabled = false }) {
    this.id = id;
    this.className = className;
    this.isDisabled = isDisabled;
    this.element = null; // DOM-элемент, который будет создан при
    рендере
  }

  // Метод для создания DOM-элемента. Может переопределяться в
  потомках.
  render() {
    const el = document.createElement('div');
    el.id = this.id;
    el.className = this.className;
    if (this.isDisabled) {
      el.setAttribute('disabled', 'true');
    }
    this.element = el;
    return el;
  }

  // Подключение обработчиков (например, клика, фокуса, и т.д.)
  attachEvents() {
    // Реализация в базовом классе может быть пустой или
    минимальной
  }

  // Отключение обработчиков, если нужно убрать элемент со
  страницы или изменить его состояние
  removeEvents() {
    // Аналогично, переопределяем в потомках
  }

  // Методы для управления состоянием
  enable() {
    this.isDisabled = false;
    if (this.element) {
      this.element.removeAttribute('disabled');
    }
  }
}
```



```

disable() {
  this.isDisabled = true;
  if (this.element) {
    this.element.setAttribute('disabled', 'true');
  }
}
}
}

```

Класс кнопки (наследник) представлен в листинге 5.15:

Листинг 5.15. Класс Button

```

// Button.js
import { BaseInteractiveElement } from
'./BaseInteractiveElement.js';

export class Button extends BaseInteractiveElement {
  constructor({ id, label, className = 'button', isDisabled =
false, onClick }) {
    super({ id, className, isDisabled });
    this.label = label;
    this.onClick = onClick;
  }

  // Переопределяем метод render для кнопки
  render() {
    const button = document.createElement('button');
    button.id = this.id;
    button.className = this.className; // Например, "button" или
"button button--large"
    button.textContent = this.label;
    if (this.isDisabled) {
      button.setAttribute('disabled', 'true');
    }
    this.element = button;
    this.attachEvents(); // Подключаем события сразу же при
рендере
    return button;
  }

  attachEvents() {
    if (this.element && typeof this.onClick === 'function') {
      this.element.addEventListener('click', this.onClick);
    }
  }

  removeEvents() {
    if (this.element && typeof this.onClick === 'function') {
      this.element.removeEventListener('click', this.onClick);
    }
  }
}

```

```
}  
}
```

Класс переключателя (Toggle Switch) представлен в листинге 5.16:

Листинг 5.16. Класс Toggle Switch

```
// ToggleSwitch.js  
import { BaseInteractiveElement } from  
'./BaseInteractiveElement.js';  
  
export class ToggleSwitch extends BaseInteractiveElement {  
  constructor({ id, className = 'toggle', isDisabled = false,  
    initialState = false }) {  
    super({ id, className, isDisabled });  
    this.state = initialState; // включён / выключен  
  }  
  
  render() {  
    const toggle = document.createElement('div');  
    toggle.id = this.id;  
    toggle.className = `${this.className} ${this.state ? 'toggle-  
-on' : 'toggle--off'}`;  
    if (this.isDisabled) {  
      toggle.setAttribute('disabled', 'true');  
    }  
    this.element = toggle;  
    this.attachEvents();  
    return toggle;  
  }  
  
  attachEvents() {  
    if (this.element) {  
      this.element.addEventListener('click', () => {  
        if (!this.isDisabled) {  
          this.state = !this.state;  
          this.element.classList.toggle('toggle--on',  
this.state);  
          this.element.classList.toggle('toggle--off',  
!this.state);  
        }  
      });  
    }  
  }  
}
```

5.3.1. Использование компонентов в приложении

Допустим, у нас есть основная страница App.js, где мы инициализируем компоненты (листинг 5.17):

Листинг 5.17. Рендер экземпляров классов Button и ToggleSwitch

```
// App.js
import { Button } from './Button.js';
import { ToggleSwitch } from './ToggleSwitch.js';

document.addEventListener('DOMContentLoaded', () => {
  // Создаём кнопку
  const myButton = new Button({
    id: 'submitButton',
    label: 'Отправить',
    className: 'button button--primary',
    onClick: () => {
      console.log('Данные отправлены!');
    }
  });

  // Создаём переключатель
  const myToggle = new ToggleSwitch({
    id: 'themeToggle',
    initialState: false,
    className: 'toggle toggle--theme'
  });

  // Рендерим компоненты на страницу
  const appRoot = document.getElementById('root');
  appRoot.appendChild(myButton.render());
  appRoot.appendChild(myToggle.render());
});
```

Если потребуется новая функциональность для кнопки (например, анимация при клике), мы можем переопределить метод `attachEvents()` или `render()` в `Button`.

Если решим создать `IconButton`, наследуемся от `Button` и добавляем свою логику (добавление иконки и т.п.).

Включая `media`-запросы в стили по БЭМ, мы легко меняем внешний вид `button--primary` или `toggle--theme` под разные размеры экрана.

Логика JavaScript при этом не меняется, т.к. поведение классов остаётся тем же.

5.3.2. Применение БЭМ для адаптивности

Поскольку у нас есть классы `button`, `button--primary`, `toggle`, `toggle--theme`, `toggle--on` и т.п., мы можем использовать медиазапросы в CSS (листинг 5.18):

Листинг 5.18. Медиазапросы для адаптивности

```
.button {
  padding: 8px 16px;
  font-size: 16px;
}

.button--primary {
  background-color: #3f51b5;
  color: #fff;
}

/* Адаптивность */
@media (max-width: 768px) {
  .button {
    font-size: 14px;
  }
}

.toggle {
  width: 50px;
  height: 30px;
  background-color: #ccc;
  cursor: pointer;
}

.toggle--on {
  background-color: #3f51b5;
}

.toggle--off {
  background-color: #ccc;
}
```

Объяснение представленным в предыдущих пунктах элементам кода:

1. Базовый класс (родитель).

Объединяет общую логику для всех интерактивных элементов: управление состоянием `isDisabled`, методы `enable()/disable()`, рендер базовой структуры.

При этом он не знает подробностей конкретного элемента (кнопки или переключателя), следуя принципам OCP (открыт для расширения) и SRP (одна ответственность – быть «базой» для интерактивного элемента).

2. Наследники (потомки).

Каждый потомок реализует или дополняет методы родителя. Например, `Button` переопределяет метод `render()`, чтобы создать тег `<button>`, а `ToggleSwitch` создает `<div>` с дополнительным состоянием «включён/выключен».

В коде, где эти компоненты используются, они могут подменять родителя – код, ожидающий работать с `BaseInteractiveElement`, сможет корректно взаимодействовать и с кнопкой, и с переключателем.

3. Расширяемость.

Если проекту потребовался новый компонент «иконка-кнопка» (`IconButton`), он может унаследоваться от `Button` и добавить логику рендеринга иконки. Мы не меняем исходный код `Button`, а лишь создаём новый класс-потомок.

Если нужен «супер-переключатель» (допустим, с дополнительной анимацией при переключении), наследуемся от `ToggleSwitch` и переопределяем соответствующие методы, не затрагивая базу.

4. Адаптивность.

На уровне кода (JS/TS) адаптивность чаще касается поведения, например, если нам нужно отключить или изменить взаимодействие при определённых условиях (размер экрана, отсутствие мыши и т.д.).

На уровне стилей (CSS/БЭМ) мы используем медиазапросы, чтобы визуально менять размер кнопок, переключателей и других элементов. Принцип «отдельный класс – отдельный CSS» упрощает поддержку.

Таким образом, даже при уменьшении экрана кнопки и переключатели будут адаптироваться к разным размерам без изменения JavaScript-логики.

5.3.3. Ключевые моменты адаптивной и масштабируемой архитектуры

1. Базовые классы. Общий функционал (свойства, методы) сосредоточен в родительском классе (`BaseInteractiveElement`), что уменьшает дублирование кода.

2. Наследование. Специфичные элементы (кнопка, переключатель) наследуются от базового класса, переопределяют только то, что нужно для уникального поведения.

3. Инкапсуляция. Каждая реализация (`Button`, `ToggleSwitch`) содержит собственный `render()`, `attachEvents()`, не «засоряя» глобальное пространство и не требуя от других компонентов знаний о внутренней структуре.

4. БЭМ-классы. Для стилизации используем единую систему именования. Модификаторы (`button--primary`, `toggle--theme`, `toggle--on`) позволяют гибко менять внешний вид, сохраняя единообразие структуры.

5. Медиазапросы. В CSS-файлах (привязанных к классам БЭМ) добавляем адаптивные стили, не меняя при этом код компонентов.

Таким образом, использование базовых классов для интерактивных элементов, наследование для специализированных компонентов и медиазапросов в стилях БЭМ формирует архитектуру, которая легко адаптируется под новые требования и быстро масштабируется без избыточных переработок.

5.3.4. Документирование архитектуры

В завершении главы студенты должны подробно описать получившуюся архитектуру проекта:

1. Показать структуру файлов и папок.
2. Объяснить логику разбиения на модули и использование принципов БЭМ.
3. Указать, как ООП применялось при проектировании компонентов.
4. Важным элементом является диаграмма структуры проекта, показывающая связь между модулями.

6. РЕАЛИЗАЦИЯ ПРОЕКТА

Курсовая работа демонстрирует практическое применение теоретических знаний, полученных в ходе изучения дисциплины, что предполагает создание полностью функционального веб-приложения с соблюдением современных стандартов фронтенд-разработки. Работа должна быть выполнена последовательно, шаг за шагом, начиная от настройки окружения и заканчивая интеграцией всех компонентов в единое приложение.

Здесь, в том числе, реализуются:

1. Структура проекта с использованием выбранной методологии (например, БЭМ).
2. Пользовательский интерфейс с помощью HTML, CSS и JavaScript.
3. Функциональность приложения на основе выбранного фреймворка (React, Vue, Angular).
4. Подключение и работа с внешними API, реализация маршрутизации и управления состоянием.
5. Добавление анимаций и улучшение пользовательского опыта.

Важным аспектом является соблюдение принципов модульности, читаемости и адаптивности кода.

6.1. Подготовка окружения для разработки

На этом этапе студентам необходимо описать, как была подготовлена рабочая среда для реализации проекта. Основные шаги:

1. Установка Node.js и менеджера пакетов npm/yarn, необходимых для управления зависимостями и сборки проекта.

Node.js – это среда выполнения JavaScript, которая необходима для работы с современными инструментами разработки.

Перейдите на официальный сайт Node.js, скачайте и установите последнюю стабильную версию (LTS) (<https://nodejs.org/en> [1]). После установки проверьте, что Node.js и npm (менеджер пакетов, который идет в комплекте) установлены корректно. Для этого выполните в терминале команды (листинг 6.1):

Листинг 6.1. Команды для проверки установленных версий Node.js и npm

```
node -v  
npm -v
```

Эти команды покажут установленные версии Node.js и npm.

Yarn – это альтернативный менеджер пакетов, который может быть полезен для ускорения установки зависимостей. Установите Yarn глобально с помощью npm (листинг 6.2):

Листинг 6.2. Установка Yarn

```
npm install -g yarn
```

Проверьте установку (листинг 6.3):

Листинг 6.3. Проверка установки Yarn

```
yarn -v
```

2. Создание нового проекта с помощью npm init или yarn init.

Для создания нового проекта используйте команду npm init или yarn init. Откройте терминал в папке, где вы хотите создать проект. Выполните команду (листинг 6.4):

Листинг 6.4. Создание нового проекта с npm

```
npm init -y
```

или, если вы используете Yarn (листинг 6.5):

Листинг 6.5. Создание нового проекта с Yarn

```
yarn init -y
```

Флаг -y автоматически создает файл package.json с настройками по умолчанию.

Файл package.json содержит информацию о вашем проекте и его зависимостях. Откройте файл package.json и при необходимости измените поля, такие как name, version, description, author и другие. Убедитесь, что в файле указана точка входа (например, "main": "index.js").

3. Установка инструментов сборки (например, Webpack, Vite) и настройка базовой конфигурации для работы с модулями, файлами стилей и медиафайлами.

Для сборки проекта можно использовать такие инструменты, как Webpack или Vite. Для примера установим Webpack и Webpack CLI (листинг 6.6):

Листинг 6.6. Установка Webpack и Webpack CLI с помощью npm

```
npm install webpack webpack-cli --save-dev
```

или с помощью Yarn (листинг 6.7):

Листинг 6.7. Установка Webpack и Webpack CLI с помощью Yarn

```
yarn add webpack webpack-cli --dev
```

Создайте файл конфигурации `webpack.config.js` в корне проекта. Пример минимальной конфигурации (листинг 6.8):

Листинг 6.8. Конфигурация `webpack.config.js`

```
const path = require('path');
module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist'),
  },
};
```

Добавьте скрипт для сборки в `package.json` (листинг 6.9):

Листинг 6.9. Сборка в `package.json`

```
"scripts": {
  "build": "webpack"
}
```

Запустите сборку (листинг 6.10):

Листинг 6.10. Запуск сборки

```
npm run build
```

Установка Vite (альтернатива) – листинг 6.11:

Листинг 6.11. Установка инструмента Vite с помощью npm

```
npm install vite --save-dev
```

или с помощью Yarn (листинг 6.12):

Листинг 6.12. Установка инструмента Vite с помощью Yarn

```
yarn add vite --dev
```

Создайте файл конфигурации vite.config.js (опционально). Добавьте скрипт для запуска разработки и сборки в package.json (листинг 6.13):

Листинг 6.13. Скрипты для разработки и сборки Vite

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build"  
}
```

Запустите сервер разработки (листинг 6.14):

Листинг 6.14. Запуск сервера разработки

```
npm run dev
```

4. Настройка линтеров и форматтеров кода (ESLint, Prettier) для обеспечения единообразия стиля написания кода.

ESLint – это инструмент для анализа и проверки качества кода. Установите ESLint (листинг 6.15):

Листинг 6.15. Установка ESLint

```
npm install eslint --save-dev
```

Инициализируйте конфигурацию ESLint (листинг 6.16):

Листинг 6.16. Инициализация конфигурации ESLint

```
npx eslint --init
```

Следуйте инструкциям в терминале, чтобы настроить ESLint под ваш проект.

Prettier – это инструмент для автоматического форматирования кода. Установите Prettier (листинг 6.17):

Листинг 6.17. Установка Prettier

```
npm install prettier --save-dev
```

Создайте файл конфигурации `.prettierrc` в корне проекта. Пример конфигурации (листинг 6.18):

Листинг 6.18. Конфигурация `.prettierrc`

```
{  
  "semi": true,  
  "singleQuote": true,  
  "tabWidth": 2  
}
```

Добавьте скрипт для форматирования кода в `package.json` (листинг 6.19):

Листинг 6.19. Скрипт форматирования

```
"scripts": {  
  "format": "prettier --write ."  
}
```

Запустите форматирование (листинг 6.20):

Листинг 6.20. Запуск форматирования

```
npm run format
```

Чтобы избежать конфликтов между ESLint и Prettier, установите плагин `eslint-config-prettier` (листинг 6.21):

Листинг 6.21. Установка плагина `eslint-config-prettier`

```
npm install eslint-config-prettier eslint-plugin-prettier --save-dev
```

Обновите конфигурацию ESLint (`.eslintrc.json`) – листинг 6.22:

Листинг 6.22. Обновление конфигурации ESLint

```
{
```

```
"extends": ["eslint:recommended",  
"plugin:prettier/recommended"]  
}
```

6.2. Разработка пользовательского интерфейса

Студенты должны реализовать основные элементы интерфейса на основе ранее созданного прототипа. Важные аспекты:

1. Использование семантических тегов HTML для построения структуры приложения.
2. Применение современных возможностей CSS, таких как гриды и флексбоксы, для адаптивной вёрстки.
3. Настройка медиазапросов для обеспечения корректного отображения интерфейса на разных устройствах.
4. Использование таких визуальных эффектов, как тени, анимации, плавные переходы, чтобы улучшить пользовательский опыт.
5. Подключение сторонних библиотек компонентов или стилей (например, Material-UI, Bootstrap), если это предусмотрено проектом.

6.3. Реализация функциональности приложения

В этом разделе студенты описывают процесс добавления интерактивности с использованием JavaScript и фреймворка, такого как React, Vue или Angular. Ключевые этапы:

1. Создание структуры приложения, включающей основные страницы или маршруты (роутинг).
2. Разработка компонентов приложения с использованием принципов композиции и инкапсуляции логики.
3. Использование состояния для управления данными в приложении (например, React useState или Vuex).
4. Настройка обработки пользовательских событий (нажатия кнопок, ввод текста, прокрутка).
5. Реализация динамических обновлений интерфейса, таких как фильтры, сортировка и отображение данных.

6.3.1. Создание структуры приложения, включающей основные страницы или маршруты (роутинг)

1. Организация структуры проекта.

Корневая папка проекта содержит все необходимые конфигурационные файлы для сборки и запуска (описание зависимостей, скрипты для запуска сервера разработки и сборки).

Например, в React-приложении можно разместить файлы настройки сборщика (например, `webpack.config` или `vite.config`) и файл `package.json` в корне, чтобы каждый разработчик знал, где искать настройки.

Папка `src` включает исходный код приложения, структурированный по функциональным модулям:

- компоненты (переиспользуемые части интерфейса), которые могут располагаться в папке `components`;
- страницы/вьюхи (логически обособленные экраны, например, `HomePage`, `AboutPage`), которые часто оформляются в папке `pages` или `views`;
- файл/файлы маршрутизации (например, `router.js`, `app-routing.module.ts`), где перечисляются пути (URL) и связывающиеся с ними компоненты.

Например, если планируется страница «Главная» и страница «О нас», можно создать соответствующие файлы (или папки) `HomePage` и `AboutPage` в разделе `pages` или `views`. Затем в файле роутинга указывается, при каком URL следует отображать конкретную страницу.

2. Настройка роутинга в современных фреймворках.

В React обычно используется отдельная библиотека для роутинга; в ней настраиваются пути (`"/"`, `"/about"`) и назначаются компоненты, которые должны отображаться при переходе по этим путям.

Во Vue создаётся специальный объект «роутера» (`router`), в котором указываются маршруты и соответствующие им вьюхи/страницы. Далее этот объект подключается к главному приложению.

В Angular существует встроенная система маршрутизации, предполагающая создание специального «`routing`-модуля», где описываются маршруты и их связь с компонентами.

Например, во Vue-проекте можно создать файл `router.js`, внутри которого перечислить, какой компонент отвечает за корневой путь `"/"` (например, `HomePage`), а какой – за путь `"/about"` (например, `AboutPage`). Подключив этот файл к приложению, вы получите возможность переключаться между страницами без перезагрузки браузера.

6.3.2. Разработка компонентов с использованием композиции и инкапсуляции

1. Разделение интерфейса на компоненты.

При разделении каждый компонент отвечает за конкретную функцию или часть интерфейса, что упрощает поддержку и повторное использование кода. Компоненты могут вкладываться друг в друга, образуя древовидную структуру приложения.

Например, в проекте интернет-магазина можно создать компоненты Header, Footer, ProductCard, ShoppingCart и использовать их на различных страницах: «Главная», «Каталог товаров», «Корзина». Компонент ProductCard можно выводить многократно, каждый раз с разными данными о товарах.

2. Преимущества и принципы композиции.

Композиция – это метод объединения нескольких мелких, независимых по своей логике элементов в более крупные структуры для решения комплексных задач.

Композиция предполагает, что функциональность приложения (или его фрагмента) разбивается на небольшие, предельно специализированные блоки. Каждый блок (например, компонент в веб-приложении) имеет чётко очерченное назначение и интерфейс взаимодействия. Объединяя такие блоки, можно создавать более сложные структуры или целые модули приложения, при этом каждый блок продолжает оставаться автономным и может использоваться повторно.

Ниже подробнее описаны преимущества и особенности композиции:

- при разбиении кода на независимые части становится проще переиспользовать их в разных местах приложения. Например, один и тот же компонент (кнопка, форма, карточка) можно применять на разных страницах;

- проще поддерживать множество небольших блоков (компонентов), чем один монолитный модуль. Изменения в одном компоненте, как правило, не влияют напрямую на работу других, при условии, что соблюден общий договорённый интерфейс;

- благодаря композиции, общую структуру приложения можно менять (переставлять компоненты, внедрять новые, удалять ненужные), не переписывая при этом большой массив кода;

- небольшие фрагменты кода, каждый из которых отвечает за чётко определённую задачу, легче осмыслить и отлаживать, чем запутанный блок с множеством функций.

Основными принципами композиции можно выделить следующие описанные аспекты:

— каждый компонент или модуль выполняет ровно одну задачу (или один тип задач) и не пытается «уметь всё». Так достигается «разделение обязанностей»: изменения в одном компоненте не ломают другие части приложения;

— для согласованной работы в больших системах компонентам необходимо передавать данные или события. При композиционном подходе стараются создать удобный интерфейс взаимодействия (например, передача «пропов» в React-компонентах или входных параметров в Angular/Vue). Если у компонента чётко описаны входные данные (и при необходимости методы обратного вызова для событий), его можно легко встроить в любую часть приложения;

— вложенность — композиция предполагает иерархическое устройство, благодаря которому более «крупные» элементы (родительские компоненты) могут включать в себя более «мелкие» (дочерние). Таким образом, постепенно наращивается сложность и детализация: от простых компонентов-блоков к целым страницам;

— инкапсуляция — изоляция внутренней логики компонентов от внешнего окружения. Внутренние детали реализуют собственную логику (например, как именно хранится локальное состояние), но внешне компонент взаимодействует лишь через оговорённый интерфейс (параметры, события);

— сильная взаимосвязь между компонентами затрудняет переиспользование: изменение одного компонента требует корректировки во многих других. Поэтому в композиционном подходе стремятся к «слабой» связи, когда компоненты лишь обмениваются данными и не вмешиваются во внутреннюю структуру друг друга.

Представим проект «Интернет-магазин». В нём может быть несколько базовых компонентов:

1. Карточка товара (отвечает только за отображение данных одного товара).
2. Список товаров (управляет массивом карточек товаров).
3. Корзина (хранит в себе список выбранных товаров).
4. Поисковая строка (позволяет вводить ключевые слова для фильтрации).

Каждый компонент автономен:

— карточка товара не знает, в каком именно списке она находится. Она получает, например, название и цену товара как входные данные и отображает их;

— список товаров может формировать набор карточек на основе полученного массива товаров, передавая в каждую карточку необходимые данные (название, цену и т.д.);

— корзина имеет собственный механизм подсчёта итоговой суммы, но сама карточка товара не знает ничего о корзине, кроме, возможно, сигнала «товар добавили в корзину»;

— поисковая строка лишь сообщает родителю, какой фильтр нужно применить к списку товаров.

В итоге компоненты легко «собираются» вместе на странице «Каталог»: сверху строка поиска, ниже – список карточек товаров, а в шапке – иконка корзины с количеством выбранных позиций. При этом взаимодействие между компонентами сведено к передаче данных, без жёсткого связывания их внутренней логики.

Также, например, компонент `UserProfile` содержит внутри себя компонент `Avatar` (для отображения фотографии) и компонент `UserDetails` (для личной информации). Родительский компонент `UserProfile` передаёт дочерним компонентам необходимые данные (например, имя пользователя и ссылку на фото), а дочерние компоненты не влияют на внешний вид профиля вне своих границ.

6.3.3. Использование состояния для управления данными

В контексте фронтенд-разработки под «состоянием» понимается совокупность данных, на которые опирается приложение при отрисовке интерфейса и выполнении логики. Состояние может содержать, например, текущий пользовательский ввод в форму, список загруженных с сервера элементов, информацию о том, авторизован ли пользователь, или выбранные настройки отображения.

Суть управления состоянием заключается в том, что при любом изменении этих данных пользовательский интерфейс реагирует соответствующим образом, обновляясь или переотрисовываясь. Это даёт возможность разрабатывать интерактивные приложения, которые динамически отвечают на действия пользователя и внешние сигналы (данные сервера, таймеры и т.д.).

1. Локальное (внутреннее) состояние.

Локальное состояние – это данные, которые относятся к одному конкретному компоненту (или небольшому набору компонентов) и не влияют напрямую на другие части приложения.

Приведем примеры применения локального состояния:

- хранение введенных пользователем значений в форме (текст в поле, выбор в выпадающем списке и т.п.) до момента отправки;
- управление тем, показать ли всплывающее окно, развернуть ли меню, переключить ли вкладку;
- использование счётчиков или локальных флагов. Например, количество кликов, состояние чекбокса, выбранная вкладка интерфейса.

Ограничением является то, что, если данные из локального состояния нужны в других компонентах, приходится явно передавать их или выносить в более высокоуровневое (глобальное) хранилище.

Можно представить компонент «Переключатель темы» (светлая/тёмная тема) на одной конкретной странице. Если переключатель действует только на стили внутри этого компонента или его дочерних элементов, то его текущее состояние (светлая или тёмная тема) может храниться локально. При нажатии кнопки компонент меняет флаг внутри себя и перестраивает оформление. При переходе на другую страницу переключатель уже неактивен, так как его состояние не влияет на всё приложение.

2. Глобальное состояние.

Глобальное состояние предназначено для хранения данных, которые необходимы сразу в нескольких местах (компонентах) или даже повсюду в приложении. Оно обычно реализуется через некий центральный механизм, чтобы обеспечить единый «источник истины» (централизованное хранилище).

Глобальное состояние нужно использовать, если данные действительно нужны многим компонентам (например, информация о текущем пользователе, выбранном языке, авторизации). Также для уменьшения сложности передачи данных из родительских компонентов к дочерним (когда уровень вложенности достаточно глубокий) и гарантии согласованности при работе нескольких компонентов с одними и теми же данными.

Допустим, в интернет-магазине хранится список товаров в корзине. Он может быть нужен одновременно в шапке (для отображения количества или суммы товаров), на странице «Каталог» (чтобы показывать, какие товары уже добавлены), и на странице «Корзина» (для вывода подробного списка и оформления заказа). Если каждый из этих компонентов будет по-своему пытаться вести учёт, возникнет несогласованность. Глобальное состояние решает это путём создания единого места (например, «хранилище корзины»), где обновляются и хранятся все данные о покупках.

3. Асинхронное обновление данных.

В современных приложениях состояние часто меняется под воздействием внешних факторов: ответов сервера, сигналов от сторонних сервисов, внутренних таймеров и т.д. Такой сценарий называется асинхронным (происходит не в момент непосредственного вызова функции, а позже, при получении результата).

Представим приложение «Список пользователей» внутри корпоративной сети. При запуске приложения оно делает запрос к удалённому сервису. Если ответ приходит через несколько секунд, пользователь видит сообщение «Загрузка...». Когда данные получены, приложение обновляет глобальное состояние, где хранится текущий список пользователей, и автоматически перестраивает интерфейс. Если сервер недоступен, пользователь получает понятное уведомление «Ошибка при загрузке».

При работе с состоянием решите заранее, какие данные будут локальными, а какие – глобальными. Это предотвращает чрезмерное перемещение данных и избыточную сложность.

Если данные необходимы только внутри одного компонента (или ограниченного числа дочерних элементов), держите их локально. Если же эти же данные нужны повсюду (или их изменения затрагивают многие области интерфейса), используйте глобальное состояние.

При необходимости обращайтесь к одному и тому же участку состояния, а не копируйте данные в разные места. Копии могут выйти из синхронизации, что приведёт к ошибкам.

Состояние – «сердце» приложения, поэтому регулярная проверка корректности обновлений (особенно асинхронных) важна для стабильности системы.

Ниже представлены более детальные описания двух распространённых подходов к управлению состоянием в популярных фреймворках: React useState (локальное состояние в React) и Vuex (централизованное хранилище для Vue). Эти технологии решают схожие задачи – управление данными, которые напрямую влияют на логику и интерфейс приложения, – но делают это по-разному.

4. React useState – локальное состояние.

В React любое изменение состояния (state) приводит к повторному рендерингу (перерисовке) соответствующего компонента. Хук useState – это встроенный механизм функциональных компонентов, позволяющий хранить и менять локальные данные.

Реакт-компонент «привязан» к значению состояния, а при каждом его изменении выполняется новая отрисовка компонента с учётом обновлённого state.

Как это работает:

- при первом вызове компонента создаётся переменная (например, счётчик) и функция, которая позволяет изменять эту переменную – инициализация;

- при изменении значения через специальную функцию React сравнивает новое состояние со старым, и, если есть изменения, инициирует повторный рендер – обновление;

- интерфейс «привязан» к локальным данным, поэтому при изменении состояния отображение компонента корректируется автоматически – реактивность.

Типичные сценарии использования:

- хранение значений, введённых пользователем (текущая строка поиска, заполнение полей регистрации);

- управление флагами состояния (открыто ли окно, каково текущее число кликов?) – для переключателей, счётчиков, модальных окон;

- если в компонент нужно загрузить какие-то данные из сети только для него одного, они тоже могут находиться в локальном состоянии.

5. Vuex – централизованное хранилище во Vue.

Vuex представляет собой шаблон и библиотеку для управления состоянием в приложениях на Vue. Он создавался для Vue 2, но может использоваться и во Vue 3 (хотя в более новых проектах часто выбирают Pinia). Принцип в том, что Vuex предоставляет единый «Store» – централизованное хранилище данных, к которому может обращаться любой компонент Vue-приложения.

Как это работает:

State – объект, в котором хранится всё «главное» состояние приложения (например, данные пользователя, настройки, списки товаров и т.п.).

Getters – вычисляемые свойства, которые позволяют производить дополнительные вычисления над state (например, подсчитывать общее количество товаров в корзине) и выдавать готовый результат компонентам.

Mutations – единственный способ «официально» изменить состояние: они получают текущее состояние и полезную нагрузку (payload), затем модифицируют state в соответствии с правилами.

Actions – служат для выполнения асинхронных операций (например, запросов к серверу), а затем вызывают необходимые мутации. Этот слой отделяет бизнес-логику (вызовы API) от непосредственных изменений состояния.

Modules – при масштабировании приложения состояние и логику Vuex можно разбивать на отдельные модули (например, user, cart, settings), что делает проект более структурированным.

Типичные сценарии использования:

1. Авторизация – данные о текущем пользователе (например, имя, токен для доступа, роли). Любая часть приложения может проверить, авторизован ли пользователь.

2. Корзина товаров – хранение списка выбранных позиций; при добавлении нового товара из каталога, при переходе на страницу корзины или при оформлении заказа все компоненты обращаются к одному и тому же хранилищу.

3. Настройки приложения, например, языковые настройки, предпочитаемая тема, которые нужно показывать/использовать на многих страницах.

Vuex применяют, когда нескольким компонентам нужны одни и те же данные (например, корзина товаров, список авторизованных пользователей, глобальные настройки) – общий state.

Когда необходимо хранить логику, связанную с изменением этих данных (валидация, асинхронные запросы к серверу) в одном месте вместо того, чтобы «раскидывать» её по многим компонентам – централизованная логика.

Vuex позволяет легко понимать, откуда возникло изменение состояния (через какую мутацию), что упрощает отладку – отслеживаемость.

6. Сравнение и рекомендации.

Если приложение небольшое и многие данные нужны лишь в одном компоненте, достаточно локального состояния (в React – useState, во Vue – обычный data или Composition API).

Если проект растёт и данные становятся «общими» для разных разделов, стоит подумать о глобальном хранилище (в React – Redux, Context API или другие решения; во Vue – Vuex или Pinia).

Разграничивайте чётко, какие данные действительно глобальные, а какие – локальные. Избегайте смешения, не помещайте в глобальное хранилище данные, которые нужны только в одном компоненте.

React useState – удобный инструмент для локального управления состоянием; хорош для небольших функциональных компонентов, не требующих комплексного взаимодействия с другими блоками приложения.

Vuex – шаблон и библиотека для централизованного управления состоянием во Vue-приложениях. Позволяет хранить в одном месте важные данные, которые нужны многим компонентам, и обеспечивает прозрачное ведение асинхронной логики.

Решение о выборе локального (React useState) или глобального (Vuex) способа хранения данных во многом зависит от масштаба приложения и требований к обмену информацией между компонентами. В большинстве реальных проектов используется комбинация: части состояния (вспомогательные/временные) остаются локальными, а ключевые глобальные данные уносятся в общий «Store».

Основная идея – выбирать инструмент по задаче: если требуется делиться данными между многими компонентами и обеспечивать предсказуемый способ их изменения, глобальное хранилище (Vuex, Redux и пр.) крайне полезно. Если же задача локальная (данные, которые не влияют на другие части приложения), стоит ограничиться локальным состоянием (React useState, Vue Composition API и т.д.).

6.3.4. Настройка обработки пользовательских событий

В контексте веб-приложений под «событиями» понимаются действия пользователя (клик, ввод, прокрутка и т.п.) или системные сигналы, которые необходимо обрабатывать для изменения состояния интерфейса, выполнения логики или взаимодействия с внешними сервисами. Все современные фреймворки (React, Vue, Angular) предоставляют механизм привязки функций-обработчиков (методов) к соответствующим событиям.

Например, если пользователь щёлкает по кнопке «Добавить в корзину», компонент может отреагировать на это событие, вызвав функцию, которая обновит состояние корзины (например, добавит соответствующий товар) и покажет уведомление.

При обработке частых событий (например, при вводе каждого символа в поле поиска или при быстрой прокрутке страницы) целесообразно использовать техники уменьшения нагрузки (дебаунсинг или троттлинг), чтобы не вызывать логику обновления слишком часто и не снижать производительность приложения.

Например, если пользователь быстро вводит текст в строке поиска, приложение может обновлять результаты не на каждую букву, а лишь спустя определённое время после последнего нажатия клавиши, что даёт более плавное взаимодействие.

1. Нажатия кнопок (Click).

Нажатия кнопок – одно из наиболее распространённых событий. Чаще всего оно используется, чтобы запустить определённую логику (например, изменение локального состояния, отправку формы или переключение темы оформления).

Привязка обработчика. В большинстве фреймворков достаточно указать атрибут (или директиву), связывающий метод компонента с событием нажатия.

Локальное обновление. При нажатии на кнопку может изменяться значение каких-либо переменных (например, счётчика), после чего фреймворк автоматически обновляет отображение.

Вызов внешних операций. Клики могут вызывать асинхронные запросы к серверу (для сохранения данных или получения новой информации), переключать страницы (роутинг) и т.д.

Доступность (Accessibility). Рекомендуется использовать элементы, предназначенные для кнопок, чтобы обеспечить правильное поведение для клавиатурных пользователей и вспомогательных технологий.

2. Ввод текста (Input).

Обработка событий ввода (input, change и т.п.) необходима при работе с полями формы и любыми элементами, где пользователь может оставлять текстовую, числовую или иную информацию.

Привязка значения. Во фреймворках обычно реализована концепция «управляемых» или «реактивных» полей ввода, при которой текущее содержимое поля напрямую связывается с переменной (или свойством) в компоненте.

Автоматическое обновление. При вводе очередного символа фреймворк самостоятельно обновляет связанное с полем значение, а в интерфейсе автоматически отражаются соответствующие изменения (например, отображение ошибок валидации или автоматическая фильтрация списка).

Двусторонняя привязка. В некоторых фреймворках предусмотрена возможность двусторонней синхронизации: изменения в переменной сразу отображаются в поле, а изменения в поле сразу меняют переменную в компоненте.

Валидация и форматирование. Событие ввода часто используется для немедленной проверки корректности введенных данных (например, корректный формат электронной почты) или для применения форматирования (подстановка тире, преобразование к заглавному регистру и т.д.).

3. Прокрутка (Scroll).

Отслеживание события прокрутки может потребоваться, чтобы реализовывать логику бесконечной подгрузки (infinite scroll), показывать анимации по мере появления элементов, фиксировать навигационную панель при достижении определённой точки или подгружать новые данные после пролистывания страницы до конца.

Уровень прослушивания. Событие прокрутки может отслеживаться либо на уровне окна (window), либо на уровне отдельного контейнера со своей полосой прокрутки.

Частота срабатывания. Прокрутка может генерировать очень много событий в секунду, поэтому для сложных операций часто применяют механизмы троттлинга (throttle) или дебаунсинга (debounce), чтобы ограничить или отложить реакцию на каждое изменение.

Подгрузка данных. В приложениях типа социальных сетей, лент новостей или каталогов товаров нередко требуется подгружать дополнительную порцию элементов при достижении нижней границы. При обработке события прокрутки обычно проверяют текущее положение и, если пользователь подходит к концу, делают запрос к серверу за новой частью данных.

Удаление обработчика. Если событие навешано напрямую на глобальный объект (например, window), важно по окончании использования компонента снять обработчик, чтобы избежать утечек памяти и нежелательных эффектов.

4. Ключевые особенности обработки событий в React, Vue и Angular.

Обработка событий в React:

- в JSX передаются обработчики событий в виде onClick, onChange и т.д.;
- React использует SyntheticEvent, который обеспечивает кроссбраузерную совместимость, но при этом имеет схожие свойства с нативным объектом события;
- названия обработчиков в React идут с большой буквы второй части: onClick, onSubmit, onKeyDown и т.п.;
- для классовых компонентов часто нужно привязывать методы к this, чтобы корректно обрабатывать события. В функциональных компонентах эта проблема решается использованием хуков.

Обработка событий во Vue:

— директива `v-on` используется для связывания события с методом в шаблоне;

— вместо `v-on:click` можно писать `@click`;

— обычно обработчики событий определяются внутри `methods` во Vue-компонентах;

— возможность модификаторов, например, `.prevent`, `.stop` и т.д., чтобы отменять дефолтное поведение или останавливать распространение события.

Обработка событий в Angular:

— используется синтаксис в круглых скобках, например, `(click)`, `(input)`, `(ngSubmit)` и т.д.;

— методы объявляются в классе компонента TypeScript, а затем вызываются в шаблоне;

— возможность использования директив, например, `(ngModelChange)` для отслеживания изменений в `ngModel`;

— если нужно получить объект события, его можно передать как `$event`.

5. Общие рекомендации и лучшие практики.

Минимизация нагрузки. При обработке частых событий (`scroll`, `input`), желательно использовать техники снижения частоты вызова, чтобы не загружать интерфейс постоянным обновлением.

Чёткое разделение логики. Каждый компонент должен обрабатывать только те события, которые действительно относятся к его зоне ответственности. Если требуется общий эффект для всего приложения (например, отслеживание общей прокрутки), лучше вынести логику в более общее место или сервис.

Доступность. Для кнопок желательно использовать стандартный элемент «`button`», для форм – поля `input`, `textarea` или `select`, так как это обеспечивает правильную работу клавиатурной навигации и вспомогательных устройств.

Обработка ошибок. Если при вводе данных нужно проверять валидность (корректность формата, заполненность, соответствие требованиям), рекомендуется предоставлять пользователю немедленную обратную связь.

Своевременная отписка. При работе с глобальными объектами (окном или документом) или при динамическом создании/уничтожении компонентов следует удалять неактуальные обработчики. Это снижает риск утечек памяти и упрощает обслуживание кода.

Настройка обработки событий (нажатия кнопок, ввод текста, прокрутка) играет ключевую роль в создании интерактивных веб-приложений. Во всех фреймворках предусмотрены удобные механизмы для привязки логики к нужному типу события.

6.3.5. Реализация динамических обновлений интерфейса

Под динамическим обновлением интерфейса понимается ситуация, когда пользовательские действия или изменения в данных приводят к немедленной перестройке/перерисовке элемента интерфейса без перезагрузки страницы. В современных фреймворках (React, Vue, Angular) это достигается за счёт реактивного состояния: при изменении переменных и коллекций автоматически пересчитываются отображаемые данные, отражая новые условия (например, заданный пользователем фильтр или выбранную сортировку).

Ключевые моменты:

- данные, подлежащие фильтрации и сортировке, обычно хранятся в локальном или глобальном состоянии приложения, что обеспечивает их реактивное обновление;

- в случае необходимости запросы к внешним сервисам осуществляются «на лету» (например, при смене параметров фильтрации или порции сортируемых данных), и по получении ответа от сервера интерфейс перестраивается;

- все изменения (новое упорядочивание, свежие результаты фильтра и т.д.) отображаются тут же, без перезапуска или перехода на другую страницу.

1. Фильтрация данных.

Фильтрация позволяет пользователю выбирать из общего набора данных только те объекты, которые удовлетворяют заданным критериям (например, все товары дешевле определённой суммы, все статьи с определённой меткой, все заказы конкретного клиента).

Хранение критериев. Обычно критерии фильтра (по цене, по категории, по ключевому слову и т.д.) хранятся в состоянии. При изменении этих критериев триггерится обновлённый набор данных.

Механизм применения. Внутри компонента или модуля, отвечающего за отображение, вызывается функция или метод, который отбирает элементы, удовлетворяя фильтру (например, посредством перебора и сравнения полей).

Перестройка интерфейса. Поскольку фреймворк автоматически следит за изменением состояния, после обновления «фильтрованных» данных компонент формирует список только тех элементов, которые соответствуют новым условиям.

Асинхронная фильтрация. При больших объёмах информации или сложной логике фильтрации запрос может отправляться к серверу с новыми параметрами, а ответ асинхронно отображаться на экране.

Применение: поиск товаров в каталоге, фильтрация по дате, быстрый поиск по имени в списке контактов.

2. Сортировка данных.

Сортировка необходима, когда пользователь хочет упорядочить набор данных по какому-либо признаку: алфавиту, дате создания, цене, рейтингу и т.д. Это позволяет быстро ориентироваться в списках и находить нужные элементы.

Хранение признака сортировки. Приложение обычно хранит выбранный признак (поле, например, «цена» или «название») и направление (возрастание или убывание) в состоянии.

Обработка данных. При смене признака сортировки исходный набор данных (или уже отфильтрованный массив) заново упорядочивается, чаще всего с использованием стандартных методов языка (без перезагрузки страницы).

Перерисовка списка. После изменения порядка элементы в интерфейсе автоматически перестраиваются. Если сортировка предполагает запрос к серверу (например, когда невозможно загрузить весь объём данных на клиенте), то параметры сортировки передаются на сервер, и полученный ответ заменяет текущий список.

Совместное применение. Фильтрация и сортировка часто комбинируются, и приложение последовательно применяет фильтр к данным, а затем сортирует полученный результат.

Применение: упорядочивание товаров по цене или рейтингу, сортировка новостей по дате публикации, сортировка списков пользователей по имени или роли.

3. Отображение данных (списки, таблицы и т.д.).

Чтобы представить отфильтрованные или отсортированные элементы, приложения обычно выстраивают их в виде списка или таблицы. Важно обеспечить не только корректную визуализацию, но и эффективное обновление (при добавлении или удалении элементов) без избыточной перерисовки.

Реактивная связка. Каждый элемент списка или строки таблицы связан с данными, хранящимися в состоянии. При их изменении фреймворк определяет, какие участки интерфейса нужно обновить, и делает это автоматически.

Уникальные идентификаторы. Хорошей практикой является предоставление каждому элементу списка уникального ключа (или идентификатора). Это даёт фреймворку возможность обновлять конкретные строки без полного пересоздания всего набора.

Оптимизация. Если объём данных велик, может потребоваться разбиение на страницы (pagination) или механизмы виртуализации (подгрузка только видимых элементов).

Масштабируемость. В случае необходимости можно применить дополнительные приёмы, такие как группировка, вложенные списки или «древовидная» структура.

Применение: отображение каталога товаров с возможностью переключаться между «карточками» и «таблицей», демонстрация результатов поиска или списка документов, таблицы с возможностью сортировки по заголовкам столбцов.

4. Дополнительные аспекты.

Пагинация и бесконечная прокрутка:

— приложение хранит информацию о текущей странице и общем количестве страниц. Пользователь переключается между страницами (1, 2, 3 и т.д.), а после каждой смены отправляется запрос на сервер или вызывается локальная фильтрация/сортировка, ограничивающая отображаемый срез данных;

— `infinite scroll` (бесконечная прокрутка). Дополнительные данные подгружаются по мере того, как пользователь достигает конца списка. Этот подход удобен для социальных сетей и длинных лент, но требует аккуратного отслеживания позиции скролла и выполнения очередных запросов на загрузку.

Производительность и отладка:

— если фильтрация или сортировка происходит очень часто (например, при каждом вводе символа в поле поиска), целесообразно использовать механизмы оптимизации (дебаунсинг, троттлинг, кэширование результатов);

— часто фильтрация или сортировка совмещены с загрузкой данных по сети. Важно корректно обрабатывать состояния «загрузка» и «ошибка» (например, выводить индикаторы ожидания и показывать сообщения об ошибках);

— рекомендуется проверять корректность отображения при разных объёмах данных и разной комбинации фильтров и сортировки, чтобы убедиться в отсутствии конфликтов логики.

Пользовательский опыт (UX):

— поля фильтра, переключатели или кнопки сортировки должны быть ясно обозначены и удобны в использовании;

— при большом массиве данных резкое перестроение интерфейса может негативно влиять на UX. Можно применять плавные анимации или ступенчатую загрузку;

— пользователь должен понимать, какие данные сейчас отображаются: если список отфильтрован или отсортирован, это стоит явно показывать (текущий критерий сортировки, активные фильтры и т.д.).

5. Итоговые рекомендации.

Пошаговая интеграция. Вначале организуйте структуру проекта и настройте базовый роутинг. Только затем добавляйте сложные компоненты, состояние и логику интеракций.

Модульность. Создавайте небольшие, автономные компоненты с чётким функциональным назначением; это повышает удобочитаемость кода и упрощает командную разработку.

Эффективное управление состоянием. Определите, какие данные можно хранить локально в компонентах, а какие необходимо вынести в глобальное хранилище, чтобы избежать дублирования и избыточной логики.

Корректная обработка событий. Обеспечьте удобство и интуитивность пользовательского взаимодействия с приложением; избегайте чрезмерной нагрузки на систему, используя оптимизационные техники.

Динамическое обновление интерфейса. Применяйте фильтрацию, сортировку и пагинацию для удобства использования приложения при работе с большими объёмами данных. Грамотное визуальное представление и своевременная подгрузка новых элементов повышают эффективность и отзывчивость интерфейса.

Тестирование и отладка. Регулярно проверяйте работу ключевых сценариев (загрузка данных, изменение состояния, навигация по страницам), чтобы предотвратить накопление ошибок. На завершающем этапе убедитесь в корректном взаимодействии всех частей приложения.

В процессе проектирования важно учитывать логику (какие фильтры, какие поля сортировки нужны), пользовательский опыт (как удобно предоставить эти возможности) и технические ограничения (оптимизация для больших объёмов данных). Правильный подход к этим задачам делает веб-приложение максимально удобным и отзывчивым для пользователей.

6.4. Подключение серверной части и работа с API

Студенты должны реализовать взаимодействие с серверной частью приложения.

Этапы реализации включают в себя:

1. Настройка запросов к API с использованием библиотеки (например, Axios или Fetch API).
2. Обработка полученных данных и отображение их на странице.
3. Работа с REST API или GraphQL для получения и отправки данных.
4. Обработка ошибок (например, недоступность сервера, ошибки сети или некорректные данные) и вывод соответствующих уведомлений пользователю.
5. Реализация функции авторизации и аутентификации, если она предусмотрена проектом.

6.4.1. Настройка запросов к API

Основными задачами этапа являются:

- настройка базового URL и заголовков для централизованной настройки всех запросов (например, указывать базовый адрес сервера, тип содержимого и данные авторизации);
- создание функций для работы с сервером, а именно организация запросов методом GET, POST, PUT, DELETE для взаимодействия с разными ресурсами API;
- реализация логики для обработки возможных ошибок сети или сервера, чтобы корректно уведомлять пользователя.

1. Общая концепция.

Современные веб-приложения часто работают по модели «клиент-сервер», где фронтенд (интерфейс на React, Vue или Angular) отправляет запросы на сервер (через HTTP), а сервер отвечает данными в формате JSON (или другим образом). Эти данные затем обрабатываются и отображаются в интерфейсе без необходимости полной перезагрузки страницы.

Настройка запросов к API – это первый и один из самых важных этапов работы с серверной частью. Здесь студентам нужно организовать взаимодействие между фронтендом и бэкендом, используя HTTP-запросы. Рассмотрим этот процесс подробнее.

2. Механизмы отправки запросов.

Fetch API. Встроенный в большинство современных браузеров, обеспечивает базовые методы для выполнения HTTP-запросов (GET, POST, PUT, DELETE и др.). Применяется для получения данных (fetching) и отправки формы или другой информации. Позволяет обрабатывать ответы с использованием промисов (then/catch) или async/await.

Библиотеки (например, Axios). Обеспечивают более удобный синтаксис и ряд дополнительных возможностей (например, автоматическую настройку заголовков, преобразование ответов, перехватчики запросов и т.д.). Часто используются при работе с REST API, где нужно много однотипных запросов.

Установка Axios через npm (если выбрали его) – листинг 6.23:

Листинг 6.23. Установка Axios через npm

```
npm install axios
```

Установка Axios через yarn (листинг 6.24):

Листинг 6.24. Установка Axios через Yarn

```
yarn add axios
```

После установки вы сможете импортировать Axios в свои модули следующим образом (листинг 6.25):

Листинг 6.25. Импорт Axios

```
import axios from 'axios';
```

Эти команды добавят Axios в зависимости вашего проекта и позволят вам начать работу с HTTP-запросами.

3. Создание базового клиента для API.

Чтобы избежать дублирования кода и упростить работу с API, рекомендуется создать отдельный файл для настройки базового клиента. Это особенно полезно, если у вас много запросов.

Создайте отдельный файл (например, `api.js`), где будете настраивать базовые параметры для запросов (базовый URL, заголовки и т.д.).

Пример для Fetch API (листинг 6.26):

Листинг 6.26. Получение данных через Fetch API

```
// Пример функции для получения данных через Fetch API
export const getData = async () => {
  try {
    const response = await fetch('https://example.com/api/data',
    {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        // При необходимости можно добавить заголовок
авторизации:
        // 'Authorization': 'Bearer <token>'
      }
    });
    if (!response.ok) {
      throw new Error(`Ошибка сети: ${response.status}`);
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Ошибка при получении данных:', error);
    // Дополнительная обработка ошибок
    throw error;
  }
};
```

Пример для Axios (листинг 6.27):

Листинг 6.27. Получение данных через Axios

```
import axios from 'axios';

// Создаем экземпляр с базовыми настройками
const api = axios.create({
  baseURL: 'https://example.com/api', // базовый URL сервера
  headers: {
    'Content-Type': 'application/json',
    // Можно добавить заголовок авторизации, если требуется:
    // 'Authorization': 'Bearer <token>'
  }
});
```

```
// Пример функции для получения данных
export const getData = async () => {
  try {
    const response = await api.get('/data'); // запрос на адрес
    https://example.com/api/data
    return response.data;
  } catch (error) {
    console.error('Ошибка при получении данных:', error);
    // Дополнительная обработка ошибок
    throw error;
  }
};
```

6.4.2. Обработка полученных данных и отображение на странице

После получения ответа от сервера следующим этапом является обработка полученных данных и их отображение на странице. Этот этап включает следующие шаги:

1. Парсинг и валидация данных. После успешного выполнения запроса необходимо убедиться, что данные получены в нужном формате (например, JSON) и соответствуют ожидаемой структуре. Если данные невалидны, их следует обработать соответствующим образом (например, вывести сообщение об ошибке).

2. Обновление состояния приложения. В приложениях на React, Vue или Angular данные обычно сохраняются в состоянии (state) компонента или хранилище. Это позволяет автоматически перерендеривать интерфейс при изменении данных.

3. Динамическое отображение. С помощью методов перебора (например, `map` в React или директивы циклов в Vue/Angular) данные выводятся в виде списка, таблицы или других элементов интерфейса. При этом можно добавить обработку кликов, фильтрацию или сортировку для улучшения взаимодействия с пользователем.

4. Обработка ошибок. Если сервер возвращает ошибку или данные не соответствуют ожидаемому формату, необходимо уведомить пользователя, например, через всплывающее сообщение или специальный элемент интерфейса.

Пример React-компонента для реализации авторизации и получения данных с сервера с помощью Fetch API (на JSX) – листинг 6.28:

Листинг 6.28. Авторизация и получение данных с помощью Fetch API

```
import React, { useState } from 'react';

function App() {
  // Состояния для хранения токена, ошибок, полученных данных
  const [token, setToken] =
useState(localStorage.getItem('authToken') || '');
  const [loginError, setLoginError] = useState('');
  const [data, setData] = useState([]);
  const [dataError, setDataError] = useState('');

  // Функция для авторизации пользователя
  async function loginUser(credentials) {
    try {
      const response = await
fetch('https://example.com/api/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(credentials)
    });
      if (!response.ok) {
        throw new Error(`Ошибка авторизации:
${response.status}`);
      }
      const result = await response.json();
      return result.token;
    } catch (error) {
      console.error('Ошибка авторизации:', error);
      throw error;
    }
  }

  // Обработчик отправки формы логина
  function handleLogin(event) {
    event.preventDefault();
    const username = event.target.username.value;
    const password = event.target.password.value;
    loginUser({ username, password })
      .then((receivedToken) => {
        setToken(receivedToken);
        localStorage.setItem('authToken', receivedToken);
        setLoginError('');
      })
      .catch(() => {
        setLoginError('Неверный логин или пароль.');
```

```

    async function fetchData() {
      try {
        const response = await
fetch('https://example.com/api/data', {
          method: 'GET',
          headers: {
            'Content-Type': 'application/json',
            ...(token && { 'Authorization': `Bearer ${token}` })
          }
        });
        if (!response.ok) {
          throw new Error(`Ошибка получения данных:
${response.status}`);
        }
        const result = await response.json();
        if (!Array.isArray(result)) {
          throw new Error('Неверный формат данных');
        }
        setData(result);
        setDataError('');
      } catch (error) {
        console.error('Ошибка загрузки данных:', error);
        setDataError('Не удалось загрузить данные. ' +
error.message);
      }
    }

    return (
      <div>
        <h1>Вход в систему</h1>
        {!token ? (
          <form onSubmit={handleLogin}>
            <label htmlFor="username">Логин:</label>
            <input type="text" id="username" name="username"
required />
            <br />
            <label htmlFor="password">Пароль:</label>
            <input type="password" id="password" name="password"
required />
            <br />
            <button type="submit">Войти</button>
            {loginError && <div style={{ color: 'red'
}}>{loginError}</div>}
          </form>
        ) : (
          <div>
            <p>Вы авторизованы.</p>
            <button onClick={fetchData}>Получить данные</button>
            {dataError && <p style={{ color: 'red'
}}>{dataError}</p>}
          </div>
        )
      </div>
    )
  )
}

```

```

        {data.map((item) => (
            <li key={item.id}>{item.name}</li>
        ))}
    </ul>
</div>
    )}
</div>
);
}

export default App;

```

JSX (JavaScript XML) – это синтаксическое расширение для JavaScript, которое позволяет писать код, похожий на HTML, прямо в JavaScript-файлах. Он используется в React для описания структуры пользовательского интерфейса. При сборке приложения код JSX компилируется (например, с помощью Babel) в вызовы функций вроде `React.createElement()`, которые создают объекты, описывающие элементы интерфейса. Это упрощает чтение и написание кода, поскольку позволяет легко комбинировать разметку с логикой приложения.

Пример реализации в React с использованием Axios (листинг 6.29):

Листинг 6.29. Обработка и получение данных с помощью Axios

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

function DataDisplay() {
    // Состояние для хранения данных и ошибок
    const [data, setData] = useState([]);
    const [error, setError] = useState(null);

    // Получение данных при монтировании компонента
    useEffect(() => {
        axios.get('https://example.com/api/data')
            .then(response => {
                // Проверяем, что данные пришли в нужном формате
                if (response.data && Array.isArray(response.data)) {
                    setData(response.data);
                } else {
                    throw new Error('Неверный формат данных');
                }
            })
            .catch(err => {
                console.error('Ошибка при получении данных:', err);
                setError('Не удалось загрузить данные');
            });
    });
}

```

```

    });
  }, []);

  // Отображение данных или сообщения об ошибке
  return (
    <div>
      <h1>Список данных</h1>
      {error ? (
        <p>{error}</p>
      ) : (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}

export default DataDisplay;

```

6.4.3. Работа с REST API или GraphQL для получения и отправки данных

1. Выбор технологии и библиотеки:

— REST API. Используйте стандартные HTTP-методы (GET, POST, PUT, DELETE). Для отправки запросов можно применять встроенный Fetch, библиотеку Axios или аналогичные инструменты;

— GraphQL. Подходит для выборочного получения данных, когда клиенту нужны лишь определённые поля. Используйте специализированные библиотеки (например, Apollo Client или Relay) для работы с запросами и мутациями.

2. Основные этапы реализации:

— анализ API; изучите документацию сервера: конечные точки (endpoints) для REST или схему (schema) для GraphQL;

— настройка клиента; настройте базовые параметры для отправки запросов (базовый URL, заголовки, авторизация).

3. Реализация запросов.

Для REST:

— реализуйте методы для получения данных (GET), создания (POST), обновления (PUT/PATCH) и удаления (DELETE);

— обрабатывайте ответы сервера, используя асинхронные функции (async/await или промисы).

Для GraphQL:

— напишите запросы (queries) для получения данных и мутации (mutations) для изменения;

— организуйте работу с кэшированием данных и обновлением состояния клиента после выполнения операций.

4. Обработка ошибок.

Реализуйте обработку ошибок (например, через блок try/catch или обработчики ошибок у Axios) для информирования пользователя о проблемах.

5. Тестирование.

Проверьте корректность работы запросов, используя инструменты отладки (например, Postman для REST или GraphiQL для GraphQL).

6. Рекомендации.

Вынесите логику работы с API в отдельный модуль или сервис, чтобы упростить поддержку и тестирование;

Интегрируйте полученные данные с системой управления состоянием (например, Redux, Vuex, Context API) для удобства работы в масштабном приложении;

Комментируйте код и описывайте процессы взаимодействия с сервером, чтобы упростить поддержку и передачу знаний коллегам или преподавателю.

Эти этапы и рекомендации помогут вам структурировать работу с серверной частью приложения, будь то через REST API или GraphQL, и обеспечить качественную интеграцию с сервером.

7. Примеры с REST API и GraphQL.

Пример работы с REST API с использованием Fetch (листинг 6.30):

Листинг 6.30. Пример с REST API

```
// Функция для получения данных (GET)
async function getData() {
  try {
    const response = await fetch('https://api.example.com/data');
    if (!response.ok) {
      throw new Error(`Ошибка: ${response.status}`);
    }
    const data = await response.json();
    console.log('Полученные данные:', data);
  } catch (error) {
    console.error('Ошибка при получении данных:', error);
  }
}
```

```
// Функция для создания нового элемента (POST)
async function createData(newItem) {
  try {
    const response = await fetch('https://api.example.com/data',
{
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(newItem)
  });
    if (!response.ok) {
      throw new Error(`Ошибка: ${response.status}`);
    }
    const result = await response.json();
    console.log('Созданный элемент:', result);
  } catch (error) {
    console.error('Ошибка при создании элемента:', error);
  }
}

// Пример вызова функций
getData();
createData({ name: 'Новый элемент', value: 42 });
```

Пример работы с GraphQL с использованием Apollo Client (React) – листинг 6.31:

Листинг 6.31. Пример с GraphQL

```
import React from 'react';
import {
  ApolloClient,
  InMemoryCache,
  ApolloProvider,
  useQuery,
  gql,
  useMutation
} from '@apollo/client';

// Создание клиента Apollo
const client = new ApolloClient({
  uri: 'https://graphql.example.com/graphql',
  cache: new InMemoryCache()
});

// Запрос для получения данных
const GET_ITEMS = gql`
  query GetItems {
```

```

    items {
      id
      name
      value
    }
  }
`;

// Мутация для добавления нового элемента
const ADD_ITEM = gql`
  mutation AddItem($name: String!, $value: Int!) {
    addItem(name: $name, value: $value) {
      id
      name
      value
    }
  }
`;

// Компонент для отображения списка элементов
function ItemsList() {
  const { loading, error, data } = useQuery(GET_ITEMS);

  if (loading) return <p>Загрузка...</p>;
  if (error) return <p>Ошибка: {error.message}</p>;

  return (
    <ul>
      {data.items.map(item => (
        <li key={item.id}>{item.name}: {item.value}</li>
      ))}
    </ul>
  );
}

// Компонент для добавления нового элемента
function AddItem() {
  const [addItem, { loading, error }] = useMutation(ADD_ITEM);

  const handleSubmit = event => {
    event.preventDefault();
    const form = event.target;
    const name = form.name.value;
    const value = parseInt(form.value.value, 10);

    addItem({ variables: { name, value } });
    form.reset();
  };

  if (loading) return <p>Отправка данных...</p>;
  if (error) return <p>Ошибка: {error.message}</p>;

```

```

    return (
      <form onSubmit={handleSubmit}>
        <input name="name" type="text" placeholder="Название"
required />
        <input name="value" type="number" placeholder="Значение"
required />
        <button type="submit">Добавить элемент</button>
      </form>
    );
  }

// Основной компонент приложения
function App() {
  return (
    <ApolloProvider client={client}>
      <div>
        <h2>Список элементов</h2>
        <ItemsList />
        <h2>Добавить новый элемент</h2>
        <AddItem />
      </div>
    </ApolloProvider>
  );
}

export default App;

```

6.4.4. Обработка ошибок и уведомления пользователя

1. Виды ошибок.

Сетевые (нет доступа к серверу), которые возникают при отсутствии интернет-соединения или недоступности сервера.

Ответы сервера (4xx, 5xx), указывающие на проблемы на стороне клиента (например, неверный запрос 400) или сервера (например, внутренняя ошибка 500).

Ошибки в формате данных, когда сервер возвращает неожиданный формат или клиент не может его обработать.

2. Основные подходы к обработке ошибок.

Проверка статуса ответа. При выполнении HTTP-запросов важно проверять статус ответа. Даже если соединение установлено, сервер может вернуть статус ошибки (например, 404 или 500).

Использование `try/catch`. Асинхронные операции, такие как запросы с использованием `Fetch API`, заключаются в блок `try/catch`. Это позволяет перехватывать ошибки, возникающие как при выполнении запроса, так и при обработке ответа.

Вывод уведомлений. При возникновении ошибки необходимо уведомлять пользователя об ошибке в понятной форме – это может быть всплывающее окно, сообщение в интерфейсе или изменение визуальных элементов.

Логирование ошибок. Для последующего анализа ошибок их часто логируют в консоль или отправляют на сервер мониторинга, что помогает выявлять повторяющиеся проблемы.

Ниже приведён пример `React`-компонента, демонстрирующего реализацию этапа обработки ошибок. В примере используется `Fetch API` для отправки запроса к серверу, а ошибки, возникающие в процессе запроса или обработки ответа, перехватываются и выводятся пользователю (листинг 6.32):

Листинг 6.32. Обработка ошибок

```
import React, { useState } from 'react';

function ErrorHandlingExample() {
  // Состояние для хранения данных и сообщения об ошибке
  const [data, setData] = useState(null);
  const [error, setError] = useState('');

  // Функция для загрузки данных с сервера
  async function fetchData() {
    try {
      // Сброс предыдущей ошибки
      setError('');

      // Отправка запроса на сервер
      const response = await
fetch('https://example.com/api/data');

      // Проверка статуса ответа: если статус не в диапазоне 200-
299, выбрасываем ошибку
      if (!response.ok) {
        throw new Error(`Ошибка сервера: ${response.status}
${response.statusText}`);
      }

      // Преобразование ответа в JSON
      const jsonData = await response.json();

      // Обновление состояния с полученными данными
      setData(jsonData);
    }
  }
}
```

```

    } catch (err) {
      // Логирование ошибки в консоль для отладки
      console.error('Ошибка при выполнении запроса:', err);

      // Установка понятного сообщения об ошибке для пользователя
      setError('Не удалось загрузить данные. Проверьте
подключение к серверу или повторите попытку позже.');
```

```

    }
  }

  return (
    <div>
      <h1>Загрузка данных с сервера</h1>
      <button onClick={fetchData}>Загрузить данные</button>

      {/* Вывод уведомления об ошибке, если она возникла */}
      {error && <div style={{ color: 'red', marginTop: '10px'
}}>{error}</div>}

      {/* Вывод полученных данных, если запрос выполнен успешно
*/}
      {data && (
        <div>
          <h2>Полученные данные:</h2>
          <pre>{JSON.stringify(data, null, 2)}</pre>
        </div>
      )}
    </div>
  );
}

export default ErrorHandlingExample;

```

6.4.5. Реализация авторизации и аутентификации

Определим разницу между двумя понятиями: аутентификацией и авторизацией.

Аутентификация – это процесс проверки подлинности пользователя. Пользователь предоставляет учетные данные (например, логин и пароль), и сервер проверяет, корректны ли они. При успешной аутентификации сервер обычно возвращает токен (например, JWT), который подтверждает личность пользователя.

Авторизация – после аутентификации система определяет, к каким ресурсам или функциям пользователь имеет доступ. Токен, полученный при аутентификации, используется в последующих запросах для проверки прав пользователя.

1. Общая схема.

При необходимости защищать определённые маршруты или функции приложения может быть реализована система аутентификации. Классическая схема – хранение токена (например, JWT) после входа в систему. При последующих запросах этот токен включается в заголовок авторизации, и сервер проверяет валидность.

API для логина – обычно сервер предоставляет endpoint, куда отправляются учетные данные методом POST. В случае успешной проверки сервер возвращает токен.

Токен можно сохранять в localStorage, sessionStorage или куках. При этом следует учитывать вопросы безопасности (например, защита от XSS-атак).

При последующих запросах к серверу токен добавляется в заголовок (например, Authorization: Bearer <token>), что позволяет серверу идентифицировать пользователя и проверять его права.

2. Примеры механизмов.

JWT (JSON Web Tokens). После проверки логина и пароля сервер выдаёт JWT-токен, который клиент хранит (например, в localStorage) и отправляет в каждом запросе.

Сессии (Sessions). Сервер устанавливает сессию, а браузер сохраняет cookie с идентификатором. Это более традиционный подход, часто требующий привязки к домену.

OAuth. Используется при интеграции со сторонними сервисами, где аутентификация переносится на внешний сервис.

Ниже представлен пример React-компонента, реализующего авторизацию с помощью Fetch API (листинг 6.33):

Листинг 6.33. Авторизация с помощью Fetch API

```
import React, { useState } from 'react';

function AuthExample() {
  // Состояния для хранения токена, ошибок и полей формы
  const [token, setToken] =
    useState(localStorage.getItem('authToken') || '');
  const [loginError, setLoginError] = useState('');
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');

  // Функция для аутентификации пользователя
  async function loginUser(credentials) {
    try {
```

```

    const response = await
fetch('https://example.com/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(credentials)
});
    if (!response.ok) {
        // Если сервер возвращает ошибку (например, 401 или 403),
выбрасываем исключение
        throw new Error('Неверные учетные данные');
    }
    const data = await response.json();
    // Предполагается, что сервер возвращает объект с полем
token
    return data.token;
} catch (error) {
    console.error('Ошибка авторизации:', error);
    throw error;
}
}

// Обработчик отправки формы логина
async function handleLogin(e) {
    e.preventDefault();
    try {
        const authToken = await loginUser({ username, password });
        setToken(authToken);
        localStorage.setItem('authToken', authToken);
        setLoginError('');
    } catch (error) {
        setLoginError('Ошибка авторизации: проверьте логин и
пароль.');
```

```

    </div>
  ) : (
    // Форма логина для ввода учетных данных
    <form onSubmit={handleLogin}>
      <div>
        <label htmlFor="username">Логин:</label>
        <input
          type="text"
          id="username"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          required
        />
      </div>
      <div>
        <label htmlFor="password">Пароль:</label>
        <input
          type="password"
          id="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
      </div>
      <button type="submit">Войти</button>
      {loginError && <p style={{ color: 'red'
}}>{loginError}</p>}
    </form>
  )}
</div>
);
}

export default AuthExample;

```

В примере (листинг 6.33):

- пользователь вводит логин и пароль;
- отправляется POST-запрос на сервер для проверки учетных данных;
- при успешном ответе токен сохраняется в состоянии и localStorage;
- в случае ошибки выводится соответствующее уведомление;
- также реализована функция выхода, которая очищает токен.

3. Защита маршрутов (роутов) во фронтенд-фреймворках.

Проверка токена. Перед переходом на защищённую страницу приложение может проверять, хранится ли у пользователя корректный токен. При его отсутствии пользователь перенаправляется на страницу входа.

Роль/уровень доступа. В сложных системах учитываются не только факт авторизации, но и роли (например, «Администратор», «Менеджер», «Пользователь»), от которых зависит, какие разделы приложения доступны.

Независимо от выбора архитектуры (REST или GraphQL), важно обеспечить:

- очевидную структуру, т.е. выделить в проекте место (сервис, модуль) для сетевых запросов, чтобы их было легко поддерживать;

- обработку ошибок, а именно предусмотреть сценарии отсутствия сети, некорректных данных, закрытых для доступа эндпоинтов;

- информирование пользователя: вовремя выводить уведомления об ошибках и прогрессе, чтобы интерфейс оставался понятным и отзывчивым;

- безопасность, в виде реализации необходимой аутентификации/авторизации, защиты конфиденциальных данных и маршрутов.

Таким образом, реализация функции авторизации и аутентификации включает следующие ключевые этапы:

- сбор данных от пользователя через форму для ввода учетных данных;

- отправку запроса на сервер с использованием Fetch API для передачи данных и получения токена;

- обработку ответа и ошибок с помощью проверки статуса ответа, вывода ошибок, если авторизация не прошла;

- хранение токена в формате сохранения токена для последующих запросов к защищённым ресурсам;

- условный рендеринг – отображение интерфейса в зависимости от статуса авторизации.

6.5. Добавление анимаций и улучшение пользовательского опыта

На этом этапе студенты должны сосредоточиться на деталях, которые делают приложение более интуитивно понятным и приятным для использования.

Примеры задач:

- добавление анимаций, основанных на CSS и JavaScript (например, появление всплывающих окон, изменение состояния кнопок);

- настройка плавных переходов между страницами или компонентами;

- улучшение производительности интерфейса с использованием lazy-loading для изображений и компонентов.

6.5.1. Роль анимаций и плавных переходов в UX

В веб-разработке анимации и переходы играют важную роль в повышении привлекательности приложения и создании позитивного пользовательского опыта. Они помогают пользователю лучше ориентироваться в интерфейсе, указывают на изменение состояния (например, при нажатии кнопки) и придают приложению «живость». В то же время важно соблюдать баланс, чтобы анимации не перегружали интерфейс и не замедляли работу приложения.

Виды анимаций:

- анимации интерфейсных элементов;
- изменение состояния кнопок (при наведении, нажатии);
- плавное появление/исчезновение всплывающих окон (диалоговых, подсказок);
- анимация списков при добавлении или удалении элементов;
- переходы между страницами (views) или компонентами;
- плавное переключение маршрутов (роутинга);
- анимация при смене вкладок или разделов приложения;
- микровзаимодействия (microinteractions);
- короткие анимированные реакции на действия пользователя (как «лайк» или «добавление в избранное»);
- визуальные подсказки (анимированное выделение полей с ошибкой, «шевеление» кнопки при неверном вводе).

1. Основные приёмы реализации анимаций.

CSS-переходы и анимации:

- большая часть простых анимаций (появление, исчезновение, изменение прозрачности, трансформация) может быть реализована средствами CSS;
- переходы (transition) позволяют плавно изменять стили элемента (цвет, размер, прозрачность и т.д.) при переключении классов или состояний;
- анимации (keyframes) дают возможность более сложного управления, определяя ключевые кадры (например, движение элемента по заданной траектории, последовательные изменения цвета и т.п.).

JavaScript-анимации – при необходимости более тонкого контроля, анимации могут быть дополнительно управляемы на уровне JavaScript (например, через библиотеки анимаций). Это даёт возможность:

- программно менять свойства в определённые моменты (собственные функции «таймлайна»);

— динамически реагировать на внешние события (прокрутка, положение курсора) и корректировать анимацию в реальном времени;

— сочетать анимацию с состоянием фреймворка, чтобы учитывать логику приложения (например, отобразить анимированную реакцию только при определённых условиях).

2. Анимация с учётом фреймворка.

В React часто используют библиотеки или встроенные утилиты (например, React Transition Group). Анимации зависят от изменения состояния (mount/unmount компонента), что позволяет создавать плавные появления и исчезновения.

Во Vue имеются собственные механизмы (transition, transition-group), позволяющие оборачивать элементы или списки в специальные контейнеры «transition» и описывать класс для появления и исчезновения.

Angular предоставляет модуль Angular Animations, основанный на методах привязки к состоянию (state, transition, trigger) и возможностях TypeScript.

6.5.2. Настройка плавных переходов между страницами или компонентами

1. Переходы в роутинге.

При навигации между страницами (разделами) может применяться анимация, позволяющая плавно «перелистывать» контент. Это удобнее, чем резкая смена экрана, так как пользователь видит логику перехода (куда «уходит» старая страница и откуда «приходит» новая).

Основные идеи:

— перехват события «смена маршрута» – фреймворки позволяют слушать переход между страницами и запускать анимацию, пока одна страница снимается с экрана, а другая появляется;

— применение ключевых кадров или классов – для входящего/исходящего состояния можно задать разные классы, которые анимируют opacity, transform и другие свойства.

2. Анимация компонентов при их появлении и скрытии.

Иногда внутри одной страницы требуется показать или скрыть определённый блок (форма, модальное окно, меню). Во многих фреймворках предусмотрены «хуки» жизненного цикла компонента или специальные инструменты (как «transition» во Vue), которые упрощают процесс анимации при монтировании/демонтировании элемента.

6.5.3. Улучшение производительности интерфейса (lazy-loading)

Lazy-loading (ленивая загрузка) – это практика, при которой ресурсы (файлы, изображения, компоненты) подгружаются не сразу при первом открытии приложения, а по мере необходимости. Это сокращает время начальной загрузки страницы и улучшает пользовательский опыт, поскольку основные элементы интерфейса показываются быстрее.

1. Lazy-loading изображений.

Загрузка «по видимости». Изображения подгружаются только тогда, когда пользователь докручивает страницу до точки, где картинка становится видимой.

Атрибуты браузера. В современных браузерах существует атрибут `loading="lazy"` для изображений, который автоматически откладывает их загрузку.

Кастомные решения. Во фреймворках и библиотеках можно настроить отслеживание позиции скrolла и подгружать картинки через программные вызовы.

2. Lazy-loading компонентов.

Некоторые части приложения (например, объёмные модули, редко посещаемые страницы) можно загружать динамически, когда пользователь действительно к ним переходит. Во фреймворках это достигается механизмом «динамического импорта» или аналогичными функциями:

Code splitting. Разделение кода на отдельные «чанки», которые подгружаются при роутинге или конкретных действиях.

Модульная структура. В Angular, например, можно объявить «ленивые» модули (lazy modules) и загружать их только при фактическом входе на соответствующий маршрут.

6.5.4. Общие рекомендации по анимациям и UX

1. Уместность и умеренность.

Анимации не должны мешать восприятию информации или замедлять интерфейс. Цель – помочь пользователю понять переходы и логику интерфейса, а не отвлекать от контента. Кратковременные и целенаправленные эффекты будут полезны; чрезмерные – могут раздражать.

2. Плавность и предсказуемость.

Избегать резких скачков. Плавное появление/исчезновение помогает глазу «следить» за элементом.

Согласованность. Если анимации используются повсеместно, важно сохранить одинаковый стиль и принципы (одинаковая длительность переходов, тип кривой анимации и т.д.), чтобы интерфейс был целостным.

3. Адаптация к производительности.

Поддержка слабых устройств. Если анимаций очень много, на некоторых устройствах возможны задержки. В ряде случаев нужно уметь отключать тяжёлые эффекты для слабых браузеров.

Оптимизация анимируемых свойств. Лучше анимировать, к примеру, transform и opacity, чем «top», «left» и «width», так как это влияет на перерисовку (reflow) и производительность.

4. Тестирование.

Кроссбраузерная проверка. Убедиться, что анимации и переходы корректно работают в основных браузерах, и при их отсутствии интерфейс остаётся функциональным.

Проверка на мобильных устройствах. Условия касания (touch), мощность процессора и частота обновления экрана могут отличаться, что повлияет на плавность.

7. ТЕСТИРОВАНИЕ И ОТЛАДКА

Тестирование является важным этапом разработки, который подтверждает работоспособность и корректность веб-приложения. Основные аспекты:

1. Написание модульных тестов для ключевых компонентов (например, с использованием Jest, Mocha).
2. Тестирование пользовательских сценариев с использованием инструментов, таких как Cypress или Selenium.
3. Проверка адаптивности приложения на различных устройствах и браузерах.
4. Отладка ошибок, выявленных в процессе тестирования.
5. Необходимо объяснить, какие методы тестирования были использованы, и показать пример тестового сценария.

Перед завершением работы студенты должны выполнить оптимизацию и подготовить приложение к развертыванию:

1. Минификация CSS и JavaScript для уменьшения размера файлов.
2. Оптимизация загрузки изображений и шрифтов.
3. Настройка серверного рендеринга, если это необходимо для SEO.
4. Упаковка проекта в готовую сборку.
5. Затем студенты описывают процесс деплоя приложения на платформу (например, Vercel, Netlify, GitHub Pages или собственный сервер).

7.1. Тестирование React-приложений

7.1.1. Юнит-тестирование (Jest)

Юнит-тесты проверяют отдельные функции или компоненты в изоляции.

Мокирование – замена реальных зависимостей (например, API-запросов) на заглушки.

Тестирование состояний – проверка, как компонент реагирует на изменения props или user-события.

Пример теста функции (листинг 7.1):

Листинг 7.1. Тест функции

```
// sum.js
```

```

export const sum = (a, b) => a + b;

// sum.test.js
test("Сумма 1 + 2 должна быть равна 3", () => {
  expect(sum(1, 2)).toBe(3); // Проверка корректности вычислений
});

Пример мокирования API:
// api.js
export const fetchData = async () => {
  const response = await fetch("https://api.example.com/data");
  return response.json();
};

// api.test.js
jest.mock("./api"); // Заменяем реальный модуль на мок

test("fetchData возвращает моковые данные", async () => {
  fetchData.mockResolvedValue({ data: "test" }); // Задаем
поведение мока
  const result = await fetchData();
  expect(result.data).toBe("test"); // Проверяем, что мок
сработал
});

```

7.1.2. Тестирование компонентов (React Testing Library)

React Testing Library (RTL) фокусируется на тестировании компонентов так, как это делает пользователь:

- поиск элементов по тексту или ролям (например, `getByRole('button')`);
- имитация событий (клики, ввод текста).

Пример теста кнопки (листинг 7.2):

Листинг 7.2. Тест кнопки

```

// Button.js
const Button = ({ onClick, children }) => (
  <button onClick={onClick}>{children}</button>
);

// Button.test.js
test("Кнопка вызывает onClick при клике", () => {
  const handleClick = jest.fn(); // Создаем мок-функцию
  render(<Button onClick={handleClick}>Click me</Button>);
  fireEvent.click(screen.getByText("Click me")); // Имитируем
клик
});

```

```
expect(handleClick).toHaveBeenCalled(); // Проверяем, что
функция вызвалась
});
```

7.1.3. Интеграционные тесты (Cypress)

Интеграционные тесты проверяют взаимодействие между компонентами или страницами. Cypress имитирует действия пользователя в реальном браузере.

Пример теста авторизации (листинг 7.3):

Листинг 7.3. Тест авторизации

```
// login.spec.js
describe("Авторизация", () => {
  it("Успешный вход в систему", () => {
    cy.visit("/login"); // Переходим на страницу
    cy.get("[data-testid=email]").type("user@test.com"); //
Вводим email
    cy.get("[data-testid=password]").type("password"); // Вводим
пароль
    cy.get("[data-testid=submit]").click(); // Нажимаем кнопку
    cy.url().should("include", "/dashboard"); // Проверяем
переход
  });
});
```

7.2. Оптимизация React-приложений

7.2.1. Мемоизация

Мемоизация – это техника оптимизации, которая позволяет избежать повторных вычислений функций или повторных рендеров компонентов при неизменных входных данных. В React это особенно важно, так как лишние рендеры замедляют работу приложения.

Основной проблемой является то, что компоненты могут перерисовываться даже тогда, когда их пропсы и состояние не изменились. Например, родительский компонент вызывает рендер дочернего, передавая одни и те же пропсы.

Решение этой проблемы:

— `React.memo` – мемоизирует компонент, сохраняя его предыдущий результат рендера и повторно используя его, если пропсы не изменились.

— `useMemo` – кэширует результат вычислений внутри компонента.

— `useCallback` – сохраняет ссылку на функцию, предотвращая ее пересоздание при каждом рендере.

Пример применения мемоизации (листинг 7.4):

Листинг 7.4. Применение мемоизации

```
// Без мемоизации компонент будет рендериться при каждом
// обновлении родителя, даже если data не изменилась
const ExpensiveComponent = React.memo(({ data }) => {
  // useMemo кэширует результат, пока data не изменится
  const computedValue = useMemo(() =>
    computeExpensiveValue(data), [data]);
  return <div>{computedValue}</div>;
});

// Тяжелая функция, которую не стоит вызывать без необходимости
const computeExpensiveValue = (data) => {
  let result = 0;
  for (let i = 0; i < 1e6; i++) result += data;
  return result;
};
```

7.2.2. Состояние на всех уровнях

Управление состоянием – ключевая часть разработки React-приложений. В зависимости от масштаба проекта, состояние может находиться на разных уровнях:

1. Локальное состояние (`useState`, `useReducer`). Используется для данных, которые нужны только внутри одного компонента (например, форма ввода).

2. Контекст (`useContext`). Позволяет передавать данные через дерево компонентов без явной передачи пропсов. Подходит для данных, которые используются в нескольких компонентах (например, тема приложения).

3. Глобальное состояние (`Redux`, `Zustand`). Необходимо для сложных приложений, где состояние используется множеством компонентов на разных уровнях (например, корзина покупок в интернет-магазине).

Пример с `Redux` (листинг 7.5):

```
// store.js
import { configureStore, createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: 0,
  reducers: {
    increment: (state) => state + 1,
  },
});

// Создаем хранилище, доступное всему приложению
export const store = configureStore({
  reducer: counterSlice.reducer,
});

// Компонент подключается к хранилищу
const Counter = () => {
  const count = useSelector((state) => state.counter);
  const dispatch = useDispatch();
  return (
    <button onClick={() =>
dispatch(counterSlice.actions.increment())}>
      {count}
    </button>
  );
};
```

7.2.3. Продвинутая навигация

React Router предоставляет инструменты для организации маршрутизации в SPA (Single Page Application):

- динамические маршруты позволяют передавать параметры через URL (например, /users/:id);
- защищенные маршруты ограничивают доступ к страницам на основе условий (например, авторизация);
- ленивая загрузка маршрутов загружает компоненты страниц только при переходе на них.

Пример защищенного маршрута (листинг 7.6):

Листинг 7.6. Защищенный маршрут

```
// Компонент-обертка для проверки авторизации
const RequireAuth = ({ children }) => {
```

```

    const { user } = useAuth(); // Предположим, что useAuth()
    возвращает данные пользователя
    return user ? children : <Navigate to="/login" />;
  };

  // Настройка роутера
  <Routes>
    <Route path="/login" element={<LoginPage />} />
    <Route
      path="/dashboard"
      element={
        <RequireAuth>
          <Dashboard />
        </RequireAuth>
      }
    />
  </Routes>;

```

7.2.4. Ленивая загрузка

Ленивая загрузка (Lazy Loading) – это стратегия, при которой ресурсы (компоненты, библиотеки) загружаются только тогда, когда они действительно нужны. Это уменьшает время начальной загрузки приложения:

- `React.lazy()` – динамически импортирует компонент;
- `Suspense` – отображает fallback (например, спиннер) во время загрузки.

Пример (листинг 7.7):

Листинг 7.7. Ленивая загрузка

```

// Без ленивой загрузки компонент загружается сразу, увеличивая
// размер основного бандла
const HeavyComponent = React.lazy(() =>
  import("../HeavyComponent"));

const App = () => (
  // Пока компонент грузится, показываем анимацию загрузки
  <Suspense fallback={<Spinner />}>
    <HeavyComponent />
  </Suspense>
);

```

7.2.5. Организация кода и архитектура

Правильная организация кода упрощает поддержку и масштабирование проекта. Популярные подходы:

1. Atomic Design. Компоненты делятся на уровни:

- атомы (кнопки, инпуты);
- молекулы (форма поиска = инпут + кнопка);
- организмы (шапка сайта = лого + навигация);
- шаблоны (макеты страниц).

2. Feature-Sliced Design. Код группируется по функциональным возможностям (например, features/cart, features/auth).

Структура проекта (листинг 7.8):

Листинг 7.8. Пример структуры проекта

```
src/
  features/
    cart/           # Все, что связано с корзиной
      components/   # Компоненты, используемые только в корзине
      hooks/        # Кастомные хуки для работы с корзиной
  shared/
    ui/            # Переиспользуемые UI-компоненты
      Button/
      Input/
```

7.3. Подготовка проекта к продакшену

7.3.1. Строгий режим React

`<React.StrictMode>` помогает выявить:

- устаревшие методы жизненного цикла;
- неожиданные сайд-эффекты;
- проблемы с анимациями.

Пример подключения (листинг 7.9):

Листинг 7.9. Подключение строгого режима

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

7.3.2. Бандлинг и динамический импорт

Сборщики (Webpack, Vite) оптимизируют код:

- Tree Shaking — удаляет неиспользуемый код;
- Code Splitting — разбивает бандл на части для параллельной загрузки.

Динамический импорт (листинг 7.10):

Листинг 7.10. Динамический импорт

```
// Загружает компонент только при переходе на страницу
const AboutPage = React.lazy(() => import("./pages/About"));
```

7.3.3. Деплой с CI/CD

CI/CD (Continuous Integration/Continuous Deployment) автоматизирует:

- запуск тестов;
- сборку проекта;
- деплой на сервер.

Пример GitHub Actions (листинг 7.11):

Листинг 7.11. Деплой проекта

```
name: Deploy
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm ci
      - run: npm run test # Если тесты провалятся, деплой не
        произойдет
      - run: npm run build
      - uses: vercel/action@v1
        with:
          token: ${ secrets.VERCEL_TOKEN }
```

7.3.4. Аналитика

Инструменты аналитики (Google Analytics, Hotjar) помогают:

- отслеживать поведение пользователей;
- находить проблемы в интерфейсе.

Пример подключения (листинг 7.12):

Листинг 7.12. Подключение аналитики

```
import ReactGA from "react-ga";
ReactGA.initialize("UA-XXXXX-Y"); // Инициализация
ReactGA.pageview(window.location.pathname); // Отслеживание
просмотров страниц
```

7.4. Дополнительные материалы

7.4.1. SSR (Server-Side Rendering)

SSR рендерит страницы на сервере, что улучшает:

- SEO (поисковые боты видят готовый HTML);
- первоначальную загрузку для медленных устройств.

Пример с Next.js (листинг 7.13):

Листинг 7.13. Рендер страницы

```
// pages/index.js
export async function getServerSideProps() {
  const res = await fetch("https://api.example.com/data");
  const data = await res.json();
  return { props: { data } }; // Данные передаются в компонент
  при рендере
}

const HomePage = ({ data }) => <div>{data}</div>;
```

7.4.2. Анимации (React Transition Group)

Библиотека react-transition-group управляет анимациями:

- плавное появление/исчезновение элементов;
- контроль времени и этапов анимации.

Пример (листинг 7.14):

```
import { CSSTransition } from "react-transition-group";

<CSSTransition
  in={isVisible}
  timeout={300}
  classNames="fade"
  unmountOnExit
>
  <div>Элемент с анимацией</div>
</CSSTransition>;
```

7.4.3. Сборка библиотеки компонентов

Для создания переиспользуемой библиотеки:

- настройка сборщика (Rollup, Webpack);
- экспорт компонентов через индексный файл;
- публикация в npm-регистр.

Конфиг Rollup (листинг 7.15):

```
// rollup.config.js
import babel from "@rollup/plugin-babel";
import resolve from "@rollup/plugin-node-resolve";

export default {
  input: "src/index.js",
  output: [
    { file: "dist/library.cjs.js", format: "cjs" }, // Для
CommonJS
    { file: "dist/library.esm.js", format: "esm" }, // Для ES-
модулей
  ],
  plugins: [
    resolve(),
    babel({ babelHelpers: "bundled" }), // Транспилиция под
старые браузеры
  ],
};
```

8. ОБЩИЕ ПОЛОЖЕНИЯ

Настоящие указания содержат требования к порядку выполнения и оформления курсовых работ (КР) по дисциплине «Фронтенд-разработка» в соответствии с Инструкцией по организации и проведению курсового проектирования (СМКО МИРЭА 7.5.1/04.И.05-18) [6].

Ниже следует ряд выдержек из данной Инструкции, полная версия которой размещена по адресу <https://www.mirea.ru/docs/177320/>. Обучающимся необходимо полностью с ней ознакомиться.

8.1. Цели и задачи курсовой работы

Основной целью курсовой работы является формирование и закрепление компетенций путем практического использования знаний, умений и навыков, полученных в рамках теоретического обучения, а также выработка самостоятельного творческого подхода к решению конкретных профессиональных задач.

Курсовая работа должна соответствовать следующим требованиям:

- соответствовать установленной структуре, а по содержанию – заданию не ее выполнение;
- быть выполненной на достаточном теоретическом уровне;
- основываться на результатах самостоятельной работы (расчеты, исследования);
- иметь обязательные самостоятельные выводы в заключении;
- иметь необходимый объем;
- быть оформленной в соответствии с установленными Инструкцией требованиями.

8.2. Структура и объем курсовой работы

Курсовая работа как письменная теоретическая работа должна иметь следующую структуру:

- титульный лист (см. Приложение 1);
- задание на курсовую работу (см. Приложение 3);
- реферат;
- содержание (оглавление);

- термины и определения (при необходимости);
- перечень сокращений и обозначений (при необходимости);
- введение;
- основная часть (главы, разделы, подразделы, пункты, подпункты);
- заключение;
- список использованной литературы (источников);
- приложение(-я) (при необходимости, не более 1/3 от общего объема работы).

Примеры заполнения титульного листа и задания на курсовую работу представлены в приложениях 4 и 6, соответственно.

Также в приложениях 2 и 5 содержатся шаблон заявления на тему курсовой работы и пример заполнения заявления.

Общий объем КР не должен превышать 50 страниц.

Реферат (рекомендуемый объем – 1-2 стр.) представляет собой краткое изложение содержания КР с основными выводами и рекомендациями, приводятся данные об объеме КР, количестве рисунков, таблиц, приложений, использованных источников. Он включает в себя также ключевые слова, которые приводятся в именительном падеже и печатаются прописными буквами, в строку, через запятые, без абзацного отступа и переноса слов, без точки в конце перечня. Текст реферата помещается с абзацного отступа после ключевых слов. Для выделения структурных частей реферата используются абзацные отступы. Перечень ключевых слов должен включать от 5 до 15 слов или словосочетаний из текста работы, которые в наибольшей мере характеризуют ее содержание и обеспечивают возможность информационного поиска [7].

В содержании (оглавлении) приводятся наименования структурных частей КР, разделов и подразделов основной части с указанием номера страницы, с которой начинается соответствующая часть, раздел (подраздел).

В перечне сокращений, условных обозначений, символов, единиц и терминов приводятся используемые в КР малораспространенные сокращения, условные обозначения, символы, единицы измерения и специфические термины. Если в перечне сокращений отсутствуют специфические термины или единицы измерения, или условные обозначения, то данный раздел в КР не включается.

Во введении (рекомендуемый объем – 1-2 стр.) дается общая характеристика КР:

- обосновывается актуальность выбранной темы;

— определяется цель работы и задачи, подлежащие решению для её достижения;

— описываются объект и предмет исследования, используемые методы и информационная база исследования, а также кратко характеризуется структура КР по разделам.

Основная часть (рекомендуемый объем – от 10 до 40 стр.) содержит материал, необходимый для достижения цели КР. Содержание основной части должно соответствовать теме, указанной в задании, и полностью ее раскрывать.

Обязательным для текста КР является логическая связь между разделами и последовательное развитие основной темы на протяжении всей работы, самостоятельное изложение материала, критический подход к изучаемым данным, проведение необходимого анализа, аргументированность выводов, обоснованность предложений и рекомендаций. Также обязательным является наличие в основной части КР ссылок на использованные источники.

9. ОФОРМЛЕНИЕ КУРСОВОЙ РАБОТЫ

Оформление работы выполняется в соответствии с требованиями ГОСТ 7.32-2017 «Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления».

Также следует обратиться к соответствующим ГОСТам:

— ГОСТ Р 2.105-2019 «Национальный стандарт Российской Федерации. Единая система конструкторской документации. Общие требования к текстовым документам» (представление текстового, табличного, формульного и иллюстративного материала);

— ГОСТ Р 7.0.100-2018 «Национальный стандарт Российской Федерации. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления» (оформление списка использованных источников);

— ГОСТ Р 7.0.5-2008 «Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления» (оформление сносок и ссылок);

— ГОСТ Р 7.0.12-2011 «Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Сокращение слов и словосочетаний на русском языке. Общие требования и правила» (использование общепринятых сокращений русских слов и сочетаний).

Ниже следует ряд выдержек из ГОСТ 7.32-2017 [7] для оформления работы, в том числе далее в подразделах 9.1 – 9.8.

При оформлении текста КР следует придерживаться следующих рекомендаций:

- формат страницы текста – А4 по ГОСТ 9327;
- ориентация страницы – книжная;
- размеры полей: левое – 30 мм, правое – 15 мм, верхнее и нижнее – 20 мм;
- шрифт – Times New Roman;
- размер шрифта – не менее 12 пт, рекомендуемый – 14 пт;
- цвет текста – черный;
- абзацный отступ – 1,25 см;
- межстрочный интервал – полуторный (1,5).

9.1. Построение отчета

Наименования структурных элементов отчета: «РЕФЕРАТ», «СОДЕРЖАНИЕ», «ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ», «ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ», «ВВЕДЕНИЕ», «ЗАКЛЮЧЕНИЕ», «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ», «ПРИЛОЖЕНИЕ» служат заголовками структурных элементов отчета.

Заголовки структурных элементов следует располагать в середине строки без точки в конце, прописными буквами, не подчеркивая. Каждый структурный элемент и каждый раздел основной части отчета начинают с новой страницы.

Основную часть отчета следует делить на разделы, подразделы и пункты. Пункты при необходимости могут делиться на подпункты. Разделы и подразделы отчета должны иметь заголовки.

Пункты и подпункты, как правило, заголовков не имеют.

Заголовки разделов и подразделов основной части отчета следует начинать с абзацного отступа 1,25 и размещать после порядкового номера, печатать с прописной буквы, полужирным шрифтом, не подчеркивать, без точки в конце. Пункты и подпункты могут иметь только порядковый номер без заголовка, начинающийся с абзацного отступа.

Если заголовок включает несколько предложений, их разделяют точками. Переносы слов в заголовках не допускаются.

9.2. Нумерация страниц отчета

Страницы отчета следует нумеровать арабскими цифрами, соблюдая сквозную нумерацию по всему тексту отчета, включая приложения. Номер страницы проставляется в центре нижней части страницы без точки.

Титульный лист включают в общую нумерацию страниц отчета. Номер страницы на титульном листе не проставляют.

Иллюстрации и таблицы, расположенные на отдельных листах, включают в общую нумерацию страниц отчета.

9.3. Нумерация разделов, подразделов, пунктов, подпунктов и книг отчета

Разделы должны иметь порядковые номера в пределах всего отчета, обозначенные арабскими цифрами без точки и расположенные с абзацного отступа. Подразделы должны иметь нумерацию в пределах каждого раздела. Номер подраздела состоит из номеров раздела и подраздела, разделенных точкой. В конце номера подраздела точка не ставится. Разделы, как и подразделы, могут состоять из одного или нескольких пунктов.

Если отчет не имеет подразделов, то нумерация пунктов в нем должна быть в пределах каждого раздела и номер пункта должен состоять из номеров раздела и пункта, разделенных точкой. В конце номера пункта точка не ставится.

Если отчет имеет подразделы, то нумерация пунктов должна быть в пределах подраздела и номер пункта должен состоять из номеров раздела, подраздела и пункта, разделенных точками.

Если раздел или подраздел состоит из одного пункта, то пункт не нумеруется.

Если текст отчета подразделяется только на пункты, они нумеруются порядковыми номерами в пределах отчета.

Пункты при необходимости могут быть разбиты на подпункты, которые должны иметь порядковую нумерацию в пределах каждого пункта.

Внутри пунктов или подпунктов могут быть приведены перечисления. Перед каждым элементом перечисления следует ставить тире. При необходимости ссылки в тексте отчета на один из элементов перечисления вместо тире ставят строчные буквы русского алфавита со скобкой, начиная с буквы «а» (за исключением букв ё, з, й, о, ч, ь, ы, ь). Простые перечисления отделяются запятой, сложные — точкой с запятой.

При наличии конкретного числа перечислений допускается перед каждым элементом перечисления ставить арабские цифры, после которых ставится скобка.

Перечисления приводятся с абзацного отступа в столбик.

9.4. Иллюстрации

Иллюстрации (чертежи, графики, схемы, компьютерные распечатки, диаграммы, фотоснимки) следует располагать в отчете непосредственно после текста отчета, где они упоминаются впервые, или на следующей странице (по возможности ближе к соответствующим частям текста отчета). На все иллюстрации в отчете должны быть даны ссылки. При ссылке необходимо писать слово «рисунок» и его номер, например: «в соответствии с рисунком 2» и т.д.

Иллюстрации, за исключением иллюстраций, приведенных в приложениях, следует нумеровать арабскими цифрами сквозной нумерацией. Если рисунок один, то он обозначается: Рисунок 1.

Иллюстрации каждого приложения обозначают отдельной нумерацией арабскими цифрами с добавлением перед цифрой обозначения приложения: Рисунок А.3.

Допускается нумеровать иллюстрации в пределах раздела отчета. В этом случае номер иллюстрации состоит из номера раздела и порядкового номера иллюстрации, разделенных точкой: Рисунок 2.1.

Иллюстрации при необходимости могут иметь наименование и пояснительные данные (подрисуночный текст). Слово «Рисунок», его номер и через тире наименование помещают после пояснительных данных и располагают в центре под рисунком без точки в конце.

9.5. Таблицы

Таблицу следует располагать непосредственно после текста, в котором она упоминается впервые, или на следующей странице.

На все таблицы в отчете должны быть ссылки. При ссылке следует печатать слово «таблица» с указанием ее номера.

Наименование таблицы, при ее наличии, должно отражать ее содержание, быть точным, кратким. Наименование следует помещать над таблицей слева, без абзацного отступа в следующем формате: Таблица Номер таблицы — Наименование таблицы. Наименование таблицы приводят с прописной буквы без точки в конце.

Если наименование таблицы занимает две строки и более, то его следует записывать через один межстрочный интервал.

Таблицу с большим количеством строк допускается переносить на другую страницу. При переносе части таблицы на другую страницу слово «Таблица», ее номер и наименование указывают один раз слева над первой частью таблицы, а над другими частями также слева пишут слова «Продолжение таблицы» и указывают номер таблицы.

Таблицы, за исключением таблиц приложений, следует нумеровать арабскими цифрами сквозной нумерацией.

Таблицы каждого приложения обозначаются отдельной нумерацией арабскими цифрами с добавлением перед цифрой обозначения приложения. Если в отчете одна таблица, она должна быть обозначена «Таблица 1» или «Таблица А.1» (если она приведена в приложении А).

Допускается нумеровать таблицы в пределах раздела при большом объеме отчета. В этом случае номер таблицы состоит из номера раздела и порядкового номера таблицы, разделенных точкой: Таблица 2.3.

9.6. Ссылки

При нумерации ссылок на документы, использованные при составлении отчета, приводится сплошная нумерация для всего текста отчета в целом или для отдельных разделов. Порядковый номер ссылки (отсылки) приводят арабскими цифрами в квадратных скобках в конце текста ссылки. Порядковый номер библиографического описания источника в списке использованных источников соответствует номеру ссылки.

9.7. Список использованных источников

Сведения об источниках следует располагать в порядке появления ссылок на источники в тексте отчета и нумеровать арабскими цифрами с точкой и печатать с абзацного отступа.

9.8. Приложения

Приложения могут включать: графический материал, таблицы не более формата А3, расчеты, описания алгоритмов и программ.

В тексте отчета на все приложения должны быть даны ссылки. Приложения располагают в порядке ссылок на них в тексте отчета.

Каждое приложение следует размещать с новой страницы с указанием в центре верхней части страницы слова «ПРИЛОЖЕНИЕ».

Приложение должно иметь заголовок, который записывают с прописной буквы, полужирным шрифтом, отдельной строкой по центру без точки в конце.

Приложения обозначают прописными буквами кириллического алфавита, начиная с А, за исключением букв Ё, З, Й, О, Ч, Ъ, Ы, Ь. После слова «ПРИЛОЖЕНИЕ» следует буква, обозначающая его последовательность. Допускается обозначение приложений буквами латинского алфавита, за исключением букв I и O.

Если в отчете одно приложение, оно обозначается «ПРИЛОЖЕНИЕ А».

Текст каждого приложения при необходимости может быть разделен на разделы, подразделы, пункты, подпункты, которые нумеруют в пределах каждого приложения. Перед номером ставится обозначение этого приложения.

Приложения должны иметь общую с остальной частью отчета сквозную нумерацию страниц.

Все приложения должны быть перечислены в содержании отчета (при наличии) с указанием их обозначений, статуса и наименования.

9.9. Листинги (программный код)

Программный код (листинги) оформляются с использованием шрифта Courier New, размер – 12 пт, междустрочный интервал – одинарный, выравнивание – по левому краю и без абзацного отступа.

Слово «Листинг» и наименование листинга через тире помещают слева перед фрагментом программного кода с заглавной буквы без абзацного отступа. Листинги нумеруются арабскими цифрами сквозной нумерацией, допускается использовать нумерацию по разделам. Название листинга печатается шрифтом основного текста работы.

Листинги, находящиеся в приложениях, нумеруются буквой соответствующего приложения и порядковым номером листинга в рамках этого приложения, разделенными точкой. Например: «Листинг Б.2 – Название листинга».

При ссылках в тексте на коды программ следует писать: «...код элемента представлен на листинге 1».

10. ПОРЯДОК ПРЕДСТАВЛЕНИЯ КУРСОВОЙ РАБОТЫ

1. Отчет о КР и остальные материалы в установленные сроки размещается в специальной зоне для загрузки курсовой работы на портале дистанционного обучения РТУ МИРЭА.

Файл отчета (структура отчета прописана в подразделе 8.2) загружается в форматах *.pdf* и *.doc/.docx/.odt*.

Файл отчета именуется строго по следующей схеме: *ФамилияИО_группа.pdf* (и *.doc/.docx/.odt* соответственно), например, *ИвановИИ_АААА-01-23.pdf*, где АААА-01-23 – номер группы.

2. Необходимо выполнить самостоятельную проверку итоговой версии КР в системе антиплагиата и результаты проверки в виде отчета приложить к курсовой работе в формате *.pdf*.

3. Подготовить презентацию для защиты в формате *.pptx*.

Структура презентации:

Слайд 1. Титульный лист.

Слайд 2. Цель и задачи курсовой работы.

Слайд 3-п Ход выполнения работы по главам. (Что сделано; Основные тезисы).

Предпоследний слайд – результаты, выносимые на защиту (коротко мотивировать решение задач и достижение цели).

Итоговый слайд «Спасибо за внимание».

ЗАКЛЮЧЕНИЕ

Курсовая работа по дисциплине «Фронтенд-разработка» представляет собой важный этап в освоении современных технологий и методологий создания пользовательских интерфейсов. В процессе выполнения работы студенты получают возможность применить теоретические знания на практике, разработав полноценный проект, который включает в себя все этапы разработки: от анализа предметной области и проектирования интерфейса до реализации, тестирования и оптимизации приложения.

В ходе выполнения курсовой работы студентами должны быть рассмотрены ключевые аспекты:

1. Формулировка темы, цели и задач курсовой работы для четкого определения направления разработки и ожидаемых результатов.
2. Анализ предметной области и обоснование выбора стека технологий, что обеспечит эффективное решение поставленных задач и, как итог, достижение поставленной цели.
3. Создание прототипа интерфейса в виде макета с использованием современных инструментов, например, такого как Figma.
4. Разработка архитектуры проекта с применением методологии БЭМ для создания масштабируемого и поддерживаемого кода.
5. Реализация проекта, включая разработку пользовательского интерфейса, подключение серверной части, работу с API и добавление анимаций для улучшения пользовательского опыта.
6. Тестирование и отладка приложения для обеспечения стабильной работы и соответствия требованиям качества.
7. Подготовка проекта к продакшену, включая оптимизацию производительности и деплой.

Заключительным и важным этапом является оформление курсовой работы в соответствии с установленными требованиями.

Результатом выполнения курсовой работы должно быть разработанное веб-приложение (система), которое соответствует поставленным изначально цели и задачам, а также созданному в ходе выполнения работы макету интерфейса.

СПИСОК ЛИТЕРАТУРЫ

1. Node.js : [сайт]. – URL: <https://nodejs.org/en> (дата обращения: 19.03.2025). – Текст: электронный.
2. Figma : [сайт]. – URL: <https://www.figma.com/> (дата обращения: 19.03.2025). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.
3. Гид по Фигме для начинающих веб-дизайнеров. – Текст : электронный // Tilda Education : официальный сайт. – 2025. – URL: <https://tilda.education/articles-figma> (дата обращения: 19.03.2025).
4. Design basics. – Текст : электронный // Figma : официальный сайт. – 2025. – URL: <https://www.figma.com/resources/learn-design/> (дата обращения: 19.03.2025).
5. Самоучитель по Figma. – Текст : электронный // Skillbox : официальный сайт. – 2025. – URL: <https://skillbox.ru/media/design/samouchitel-po-figma/> (дата обращения: 19.03.2025).
6. СМКО МИРЭА 7.5.1/04.И.05-18 Инструкция по организации и проведению курсового проектирования. – М. : РТУ МИРЭА, 2018. – 17 с.
7. ГОСТ 7.32-2017 Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М. : Стандартинформ, 2017. – 27 с.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: _____

по профилю: _____

направления профессиональной подготовки: _____

Тема: _____

Студент: _____

Группа: _____

Работа представлена к защите _____ (дата) _____ / _____ /
(подпись и ф.и.о. студента)

Руководитель: _____

Работа допущена к защите _____ (дата) _____ / _____ /
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____ / _____ /

_____ / _____ /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

М. РТУ МИРЭА. 2025 г.

Приложение 2 Заявление на тему

Заведующему кафедрой
инструментального и прикладного
программного обеспечения (ИиППО)
Института информационных технологий (ИТ)
Болбакову Роману Геннадьевичу

От студента _____

ФИО

группа

____ курс
курс

Контакт: _____

Заявление

Прошу утвердить мне тему *курсовой работы/курсового проекта* по дисциплине «Дисциплина» образовательной программы бакалавриата 09.03.04 (Программная инженерия).

Тема: _____

_____.

Приложение: лист задания на КР/КП в 2-ух экземплярах на двухстороннем листе (проект)

Подпись студента _____ / _____
подпись ФИО

Дата _____

Подпись руководителя _____ / _____
подпись Должность, ФИО

Дата _____



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине: _____

по профилю: _____

Студент: _____

Группа: _____

Срок представления к защите: _____

Руководитель: _____

Тема: _____

Исходные данные: _____

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____ /Болбаков Р. Г./, «_____» _____ 2025 г.

Задание на КР выдал: _____ / _____ /, «_____» _____ 2025 г.

Задание на КР получил: _____ / _____ /, «_____» _____ 2025 г.

Приложение 4 Пример заполнения титульного листа



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Фронтенд-разработка

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: Программная инженерия (09.03.04)

Тема: «Веб-приложение для учета расходов и доходов»

Студент: Иванов Иван Иванович

Группа: ИКБО-10-23

Работа представлена к защите _____ (дата) _____ /Иванов И.И./
(подпись и ф.и.о. студента)

Руководитель: Русяков Алексей Александрович, ст. преподаватель

Работа допущена к защите _____ (дата) _____ /Русяков А.А./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____/_____/_____
_____/_____/_____

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших
защиту)

М. РТУ МИРЭА. 2025 г.

Приложение 6 Пример заполнения листа задания



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Фронтенд-разработка

по профилю: Разработка программных продуктов и проектирование информационных систем

Студент: Иванов Иван Иванович

Группа: ИКБО-10-23

Срок представления к защите: с 26.05.2025 по 31.05.2025

Руководитель: Русяков Алексей Александрович, ст. преподаватель

Тема: «Веб-приложение для учета расходов и доходов»

Исходные данные: используемые технологии: HTML5, CSS3, JavaScript (ES6+), TypeScript, React.js, Node.js, npm, Webpack, API, JSON, Git, Среда разработки: использование Visual Studio Code с необходимыми расширениями (ESLint, Prettier, Live Server), редактор кода Visual Studio Code. Нормативный документ: Инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Провести анализ современных подходов и технологий в разработке клиентских интерфейсов с использованием фреймворков. 2. Обосновать выбор стека технологий для разработки фронтенд-приложения (React/Vue.js, TypeScript, Node.js). 3. Разработать техническое задание на создание интерактивного веб-приложения. 4. Реализовать пользовательский интерфейс с применением HTML5, CSS3 (градиенты, анимации, трансформации) и адаптивной верстки. 5. Создать компоненты приложения с использованием фреймворка, реализовав основные функции с применением ООП и TypeScript. 6. Интегрировать внешние API для получения и обработки данных в реальном времени. 7. Провести тестирование интерфейса (функциональное и юнит-тестирование). 8. Подготовить презентацию с описанием технической реализации, демонстрацией приложения и анализом выполненных задач.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____ /Болбаков Р. Г./, « _____ » _____ 2025 г.

Задание на КР выдал: _____ /Русяков А.А./, « _____ » _____ 2025 г.

Задание на КР получил: _____ /Иванов И.И./, « _____ » _____ 2025 г.

Сведения об авторах

Русяков Алексей Александрович, старший преподаватель, кафедра инструментального и прикладного программного обеспечения (ИиППО) Института информационных технологий (ИИТ)

Братусь Надежда Валерьевна, ассистент, кафедра инструментального и прикладного программного обеспечения (ИиППО) Института информационных технологий (ИИТ)